

모바일 애플리케이션 보안 취약점 분석 보고서

대상: DIVA DeepLink

기준: 전자금융기반시설 MOBILE 13 개 점검항목

작성일: 2026-08-27

분석기간: 2026-08-17 ~ 작성일 현재

작성자: 김범희

문서관리 정보

표 1. 문서 정보

구분	내용
문서명	모바일 애플리케이션 보안 취약점 분석 보고서
진단 대상	DIVA DeepLink
패키지명	jakhar.aseem.diva
앱 버전	1.0
문서 버전	1.2
진단 기준	MOBILE 13 개 점검항목 기준표
문서 상태	공공기관 제출용 버전 1.2
배포 범위	

목차

문서관리 정보.....	2
목차.....	2
1. 보고서 개요.....	4
1.1 수행 목적.....	4
1.2 수행 대상.....	4
1.3 기준 적용 원칙 및 한계.....	4
2. 경영진 요약.....	4
3. 수행 방법.....	5
3.1 분석 방법 및 절차.....	5
3.2 점검 대상 – 전자금융기반시설 MOBILE 13 개 항목.....	6
3.3 사용 도구.....	8
4. 진단 결과 요약.....	9
4.1 진단 결과 요약.....	9
4.2 총평.....	9
5. 상세 진단 결과.....	10
5.1 MOB-FIN-011 [전자금융] OS 변조 탐지 기능 적용.....	10

5.2 MOB-FIN-012 [전자금융] 악성코드 방지	12
5.3 MOB-FIN-018 [전자금융] 프로그램 무결성 검증	14
5.4 MOB-FIN-020 [전자금융] 소스코드 난독화 적용	18
5.5 MOB-FIN-021 [전자금융] 디버깅 탐지기능 적용	20
5.6 MOB-SER-008 단말기 내 중요정보 저장 방지	22
5.7 MOB-SER-009 메모리 내 중요정보 노출 방지	26
5.8 MOB-SER-020 화면 내 중요정보 평문노출 방지	29
5.9 MOB-SER-025 앱 소스코드 내 운영정보 노출 방지.....	30
5.10 MOB-SER-026 화면 강제실행에 의한 인증단계 우회 방지	32
5.11 MOB-SER-042 디버그 로그 내 중요정보 노출 방지	33
5.12 MOB-SER-043 백그라운드 화면 보호	35
5.13 MOB-SER-050 모바일 DeepLink 도용 제한여부	38
6. 종합 보안 권고.....	40
6.1 MOB-FIN-011 [전자금융] OS 변조 탐지 기능 적용	40
6.2 MOB-FIN-012 [전자금융] 악성코드 방지	41
6.3 MOB-FIN-018 [전자금융] 프로그램 무결성 검증	42
6.4 MOB-FIN-020 [전자금융] 소스코드 난독화 적용	43
6.5 MOB-FIN-021 [전자금융] 디버깅 탐지기능 적용	44
6.6 MOB-SER-008 단말기 내 중요정보 저장 방지	45
6.7 MOB-SER-009 메모리 내 중요정보 노출 방지	46
6.8 MOB-SER-020 화면 내 중요정보 평문노출 방지	47
6.9 MOB-SER-025 앱 소스코드 내 운영정보 노출 방지.....	48
6.10 MOB-SER-026 화면 강제실행에 의한 인증단계 우회 방지	49
6.11 MOB-SER-042 디버그 로그 내 중요정보 노출 방지	50
6.12 MOB-SER-043 백그라운드 화면 보호	51
6.13 MOB-SER-050 모바일 DeepLink 도용 제한여부	52
6.14 종합 개선 실행계획	52
7. 최종 결론.....	54

1. 보고서 개요

1.1 수행 목적

본 진단은 교육용 취약 안드로이드 애플리케이션인 DIVA DeepLink 를 대상으로 모바일 보안통제의 적용 여부를 점검하기 위해 수행하였다. 의뢰인이 제공한 MOBILE 13 개 점검항목을 적용하여 APK 정적 분석과 실행환경 기반 동적 분석을 실시하였으며, 확인된 취약점에 대한 개선방안을 제시하였다.

- 기존에 확보한 증거를 검토하고 유효성을 재확인하였다.
- 메모리 노출, 백그라운드 화면 보호 및 악성코드 방지 항목을 추가로 검증하였다.
- 보호통제의 적용 사실과 실효성이 객관적 증거로 확인되지 않거나 정적·동적 검증에서 보호통제가 관찰되지 않으면 취약으로 판정하고, 직접 재현 여부는 증거 수준(E1~E3)으로 별도 표시한다.

1.2 수행 대상

항목	diva_deeplink.apk
애플리케이션	DIVA DeepLink
Package Name	jakhar.aseem.diva
Version	1.0
Platform	Android
분석 방법	정적 분석 + 동적 분석
분석기간	2026-08-17 ~ 작성일 현재

1.3 기준 적용 원칙 및 한계

- 진단코드·항목명·진단설명·위험도는 사용자가 제공한 MOBILE 13 개 항목 기준표를 기준으로 한다.
- 위험도 숫자 3~5 는 제공 기준표의 값이며 CVSS Critical/High/Medium 과 직접 환산하지 않는다.
- DIVA 는 실제 전자금융 서비스 앱이 아니므로 금융 업무기능보다 모바일 단말·앱 보안통제 구현 여부를 중심으로 평가한다.
- 보호통제 적용 증거가 없고 요구 보호기능도 확인되지 않은 항목은 보수적으로 취약 판정한다. 실제 공격 성공을 직접 재현한 경우와 통제 부재에 근거한 판정은 증거 수준으로 구분한다.

2. 경영진 요약

총 13 개 점검항목 모두에서 보안통제 미흡 또는 취약 동작이 확인되어 취약으로 판정하였다. 주요 위험은 앱 변조 및 동적 분석 환경에 대한 방어 미흡, 중요정보의 저장·화면·로그·메모리 노출, 인증단계 우회와 딥링크 검증 미흡이다. 핵심 보호통제를 우선 적용하고 개선 완료 후 동일한 절차로 재점검할 필요가 있다.

표 3. 진단 결과 분포

구분	건수	비율	의미
취약	13	100.0%	보호통제 미적용·미확인 또는 취약 동작 확인
확인불가	0	0.0%	보수적 판정 원칙 적용으로 잔여 항목 없음
양호	0	0.0%	현 범위에서 양호 판정 없음
합계	13	100.0%	MOBILE 13 개 점검항목

진단 결과 분포 (MOBILE 13개 항목)

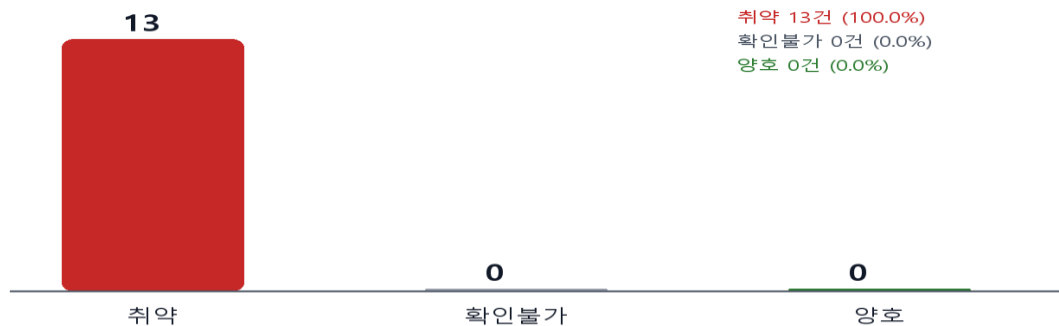


그림 0. 취약·확인불가·양호 항목 수 비교

3. 수행 방법

3.1 분석 방법 및 절차

단계 1. APK 구조 및 Manifest 분석

- APK Easy Tool 과 JADX 를 이용하여 AndroidManifest.xml, 컴포넌트(Component), 리소스(Resource), DEX 및 네이티브 라이브러리 구조를 확인하였다.

단계 2. 소스코드 및 저장정보 정적 분석

- JADX 와 AstroGrep 으로 비밀정보(Secret), 로그(Log), SharedPreferences, SQLite, 파일(File), 딥링크(DeepLink) 및 디버깅 설정을 분석하였다.

단계 3. Nox/ADB 동적 분석

- 루팅된 에뮬레이터, ADB, JDB 및 Logcat Viewer 를 이용하여 실행환경·디버깅·로그·파일·딥링크 동작을 검증하였다.

단계 4. Frida 기반 런타임 및 메모리 분석

- Frida Server 와 대상 앱의 연결 상태를 확인한 후 DIVA 프로세스를 식별하였다. 테스트 데이터를 처리한 상태에서 FridaDump3 로 프로세스 메모리를 수집하고, 화면 전환 전후에 해당 데이터가 메모리에 남아

있는지 비교하였다. Frida 는 실행 중인 앱을 분석하는 동적 분석 도구이며, FridaDump3 는 프로세스 메모리를 수집하는 도구이다.

단계 5. 변조 및 역공학 검증

- APK 추출·디컴파일·Resource 변조·재컴파일·재설치를 통해 프로그램 무결성 검증 여부를 확인한다.

단계 6. 항목별 판정

- 기준에 직접 대응되는 증거를 우선 적용하고, 보호통제 적용과 실효성이 확인되지 않으면 취약으로 판정한다. 직접 재현 여부는 E1·E2·E3 증거 수준으로 별도 구분한다.

3.2 점검 대상 – 전자금융기반시설 MOBILE 13 개 항목

항목 코드	진단항목명	진단항목 설명	위험도
MOB-FIN-011	[전자금융] OS 변조 탐지 기능 적용	OS 가 변조(루팅, 탈옥) 된 단말 이용 시 보안 위험이 증대됨에 따라, OS 변조 시 전자금융 서비스 이용가능 여부를 점검	5
MOB-FIN-012	[전자금융] 악성코드 방지	악성코드에 대한 대응이 필요함에 따라, 전자금융서비스 이용 시 악성코드 차단 및 탐지 프로세스 동작 여부를 점검	5
MOB-FIN-018	[전자금융] 프로그램 무결성 검증	변조된 프로그램이 정상실행 될 경우 악성코드가 포함되어 재배포 되는 등의 보안 위험이 존재함에 따라, 변조 프로그램 이용 시 정상 실행 가능 여부를 점검	5

항목 코드	진단항목명	진단항목 설명	위험도
MOB-FIN-020	[전자금융] 소스코드 난독화 적용	디컴파일(DeCompile) 하여 복구된 소스코드에 클래스명, 함수명 등이 평문으로 노출되어 프로그램의 구조 및 코드 흐름 파악 가능 여부를 점검	4
MOB-FIN-021	[전자금융] 디버깅 탐지기능 적용	동적 디버깅 가능 시, 프로그램 흐름 파악 등이 용이함에 따라 안티디버깅 기능 적용 여부를 점검	4
MOB-SER-008	단말기 내 중요정보 저장 방지	애플리케이션 사용 폴더 및 외부 저장소에 존재하는 파일 내 중요정보 저장 여부를 점검	5
MOB-SER-009	메모리 내 중요정보 노출 방지	이용자 단말기 메모리 영역에서 이용자 중요정보의 평문노출 여부를 점검	5
MOB-SER-020	화면 내 중요정보 평문노출 방지	이용자 단말기에서 화면 내에 이용자 중요정보의 평문 노출 여부를 점검	5
MOB-SER-025	앱 소스코드 내 운영정보 노출 방지	앱 소스코드 내 운영정보 노출 여부를 점검	5
MOB-SER-026	화면 강제실행에 의한 인증단계 우회 방지	화면강제 실행, 인증관련 파일 조작 등을 통해 인증단계 우회 가능 여부를 점검	5
MOB-SER-042	디버그 로그 내 중요정보 노출 방지	디버그 로그 내 중요정보 노출 여부를 점검	5

항목 코드	진단항목명	진단항목 설명	위험도
MOB-SER-043	백그라운드 화면 보호	실행중이던 애플리케이션이 백그라운드 상태로 진입할 때 저장되는 스냅샷 파일 내 중요정보 노출 여부를 점검	3
MOB-SER-050	모바일 DeepLink 도용 제한여부	공격자에 의해 조작된 URL 을 이용자의 사용환경(웹뷰, 애플리케이션 등)에서 접속할 때 개발자가 의도하지 않은 기능 동작 여부를 점검	5

3.3 사용 도구

도구	주요 용도
APK Easy Tool	APK 디컴파일·리패키징·Resource/Smali 구조 확인
JADX	DEX/Manifest/Java 코드 분석
AstroGrep	Secret-운영정보·취약 API 문자열 검색
Nox Player	Android Emulator 및 Root 실습 환경
ADB / PowerShell	파일·패키지·Activity·DeepLink·JDWP 동적 검증
JDB	Runtime Debugging 및 Breakpoint 검증
Logcat Viewer	런타임 로그 중요정보 노출 분석
F1ridaDump	FridaDump3 1.0.0(fridump3.exe): Frida 17.10.0 기반 Android 프로세스 메모리 수집 및 문자열 분석. Frida Server 17.10.0, Frida CLI 17.10.0, Nox Player(Android 7.1.2/x86), Nox ADB 1.0.36, Python 바이너리 검색을 보조 도구로 사용

4. 진단 결과 요약

4.1 진단 결과 요약

평가항목 ID	평가항목	진단 결과
MOB-FIN-011	[전자금융] OS 변조 탐지 기능 적용	취약
MOB-FIN-012	[전자금융] 악성코드 방지	취약
MOB-FIN-018	[전자금융] 프로그램 무결성 검증	취약
MOB-FIN-020	[전자금융] 소스코드 난독화 적용	취약
MOB-FIN-021	[전자금융] 디버깅 탐지기능 적용	취약
MOB-SER-008	단말기 내 중요정보 저장 방지	취약
MOB-SER-009	메모리 내 중요정보 노출 방지	취약
MOB-SER-020	화면 내 중요정보 평문노출 방지	취약
MOB-SER-025	앱 소스코드 내 운영정보 노출 방지	취약
MOB-SER-026	화면 강제실행에 의한 인증단계 우회 방지	취약
MOB-SER-042	디버그 로그 내 중요정보 노출 방지	취약
MOB-SER-043	백그라운드 화면 보호	취약
MOB-SER-050	모바일 DeepLink 도용 제한여부	취약

표 6. 전체 진단 결과 요약

4.2 총평

확인된 취약점은 실행환경 보호, 앱 무결성 및 역공학 방지, 중요정보 보호, 인증·외부 호출 통제의 네 영역으로 구분된다. 루팅·악성 도구·디버깅 환경 차단, 저장소·화면·로그·메모리 보호, Activity 및 딥링크 접근통제를 우선 개선해야 한다.

5. 상세 진단 결과

5.1 MOB-FIN-011 [전자금융] OS 변조 탐지 기능 적용

진단 코드	MOB-FIN-011	진단 결과	취약
진단 항목명	[전자금융] OS 변조 탐지 기능 적용	기준 위험도	5
진단 기준	OS 가 변조(루팅, 탈옥) 된 단말 이용 시 보안 위험이 증대됨에 따라, OS 변조 시 전자금융 서비스 이용가능 여부를 점검		
대응방안	- Root/Jailbreak 여부를 단일 파일·명령에만 의존하지 않고 다중 신호로 탐지한다. - 단말 무결성 신호(예: 플랫폼 무결성 API)를 서버 측 위험평가와 결합한다. - 고위험 변조 단말에서는 로그인·거래 기능을 제한하고 탐지 이벤트를 서버에 기록한다.		

상세 분석

단계 1. Root Emulator 환경 확인

Nox Player 설정에서 ROOT 기능이 활성화된 테스트 환경을 구성하였다.

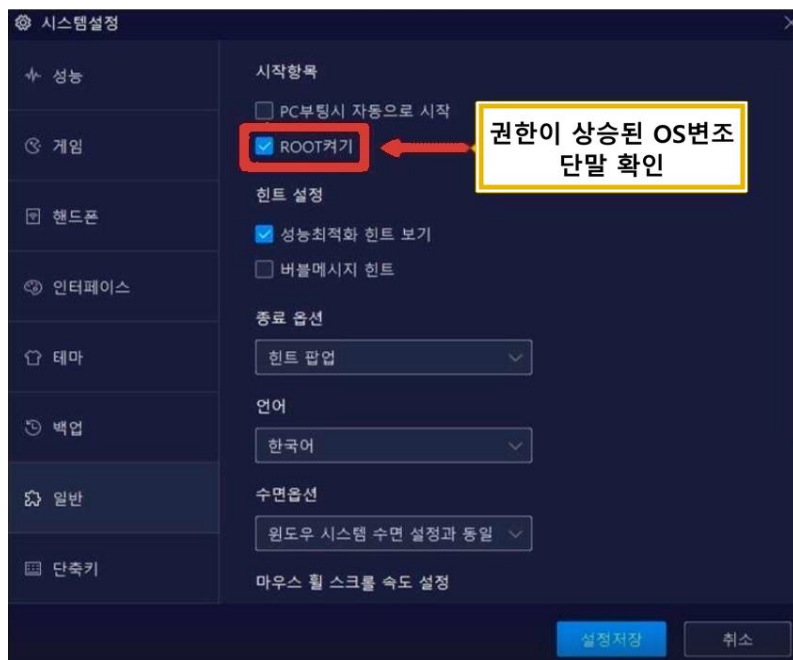


그림 1. Nox Player 에서 Root 권한이 활성화된 테스트 환경 확인

단계 2. Root 환경에서 애플리케이션 실행

Root 권한이 활성화된 단말에서 DIVA 가 실행되고 주요 화면에 접근되는지 확인하였다.



그림 2. Root 가 활성화된 Nox 환경에서 DIVA 애플리케이션 확인

단계 3. 탐지·차단 여부 판정

Root 상태임에도 별도 경고·차단 없이 앱 사용이 가능한 것으로 확인되어 OS 변조 탐지 통제가 미흡한 것으로 판정하였다.

보안 영향

Root/탈옥 단말은 앱 데이터·메모리·후킹·파일 접근이 쉬워져 인증정보 탈취 및 앱 로직 변조 위험이 증가한다.

5.2 MOB-FIN-012 [전자금융] 악성코드 방지

진단 코드	MOB-FIN-012	진단 결과	취약
진단 항목명	[전자금융] 악성코드 방지	기준 위험도	5
진단 기준	악성코드에 대한 대응이 필요함에 따라, 전자금융서비스 이용 시 악성코드 차단 및 탐지 프로세스 동작 여부를 점검		
대응방안	- 승인된 모바일 악성코드 탐지·차단 모듈 또는 연계 보안통제를 적용한다. - 앱 실행 시 알려진 위험 앱·후킹/변조 도구·의심 프로세스 등 위험 신호를 종합 평가한다. - 동적 검증에서는 검증용 테스트 패키지를 이용해 탐지·차단 프로세스가 실제 동작하는지 확인한다.		

상세 분석

단계 1. 악성코드 방지 통제 정적 확인

모바일 백신·악성 앱 탐지 SDK 적용 증거가 확인되지 않았고, 기존 Root·Debug 가능 환경에서 앱이 정상 실행된 증거가 존재한다. Frida Server·후킹 도구 등 위험 분석 환경을 탐지·경고·차단하는 통제의 적용 사실과 실효성도 확인되지 않았다.

단계 2. Frida 위험 환경 탐지·차단 검증 절차

- Nox Player 와 ADB 연결, Frida Server 버전·실행 상태, Frida CLI 의 DIVA 프로세스 식별 여부를 확인한다.
- Frida Server 실행 상태에서 DIVA 를 실행하여 경고·강제종료·기능 제한·서버 차단 이벤트를 확인한다.
- 탐지·차단 없이 앱과 주요 기능이 실행되는지 확인하고, 실제 Frida Attach 후 정상 실행 결과를 E1 취약 증적으로 기록하였다.

단계 3. 판정

세션 1 에서 Frida Attach 대상 PID 5298 을 확인했고, 세션 2 에서 FridaDump3 메모리 수집 대상 PID 3004 를 확인하였다. PID 는 앱 재실행에 따라 변경될 수 있다. Frida Server 17.10.0 이 실행 중인 Nox 에서 DIVA 프로세스에 실제 Attach 하였다. Attach 전후 앱은 경고·강제종료·기능 제한 없이 정상 동작하여 위험 도구 탐지·차단 통제 부재를 직접 확인했으므로 취약/E1 로 판정한다.

```

EVIDENCE 34 - Nox ADB conn
[ACTUAL EXECUTION] Nox / ADB connection verification
Capture time: 2026-08-27 KST

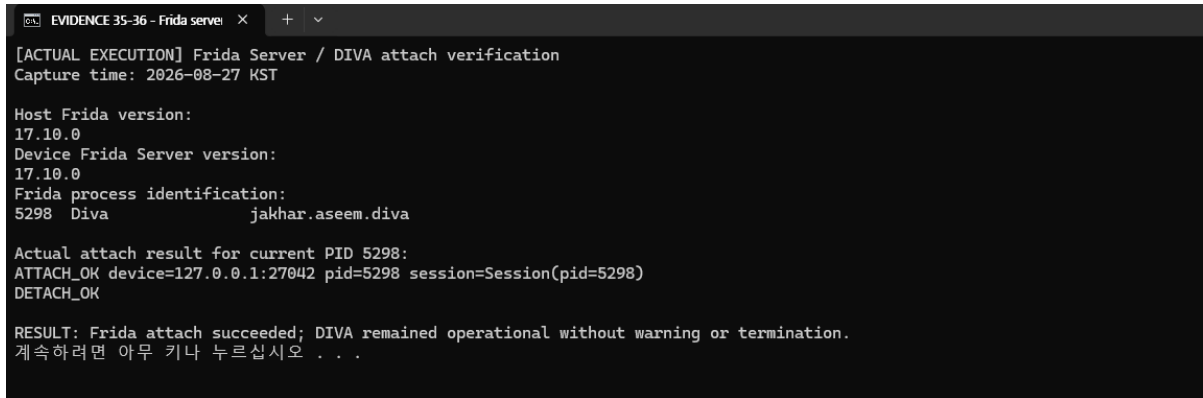
List of devices attached
127.0.0.1:62001    device product:SM-G955N model:SM-G955N device:dream2lteks

Device model:
SM-G955N
CPU ABI:
x86
Android:
7.1.2
Root identity:
uid=0(root) gid=0(root) groups=0(root),1004(input),1007(log),1011(adb),1015(sdcard_rw),1028(sdcard_r),3001(net_bt_admin),3002(net_bt),3003(inet),3006(net_bw_stats),3009(readproc)
DIVA package and PID:
package:/data/app/jakhar.aseem.diva-1/base.apk
5298

RESULT: Nox connected in device state; DIVA installed and running.
계속하려면 아무 키나 누르십시오 . . .

```

그림 34. ADB 를 이용한 실행 중인 Nox 단말 및 DIVA 연결 확인



```

EVIDENCE 35-36 - Frida server
[ACTUAL EXECUTION] Frida Server / DIVA attach verification
Capture time: 2026-08-27 KST

Host Frida version:
17.10.0
Device Frida Server version:
17.10.0
Frida process identification:
5298 Diva jakhar.aseem.diva

Actual attach result for current PID 5298:
ATTACH_OK device=127.0.0.1:27042 pid=5298 session=Session(pid=5298)
DETACH_OK

RESULT: Frida attach succeeded; DIVA remained operational without warning or termination.
계속하려면 아무 키나 누르십시오 . . .

```

그림 35. Frida Server 연결·DIVA 프로세스 식별 및 Attach 성공

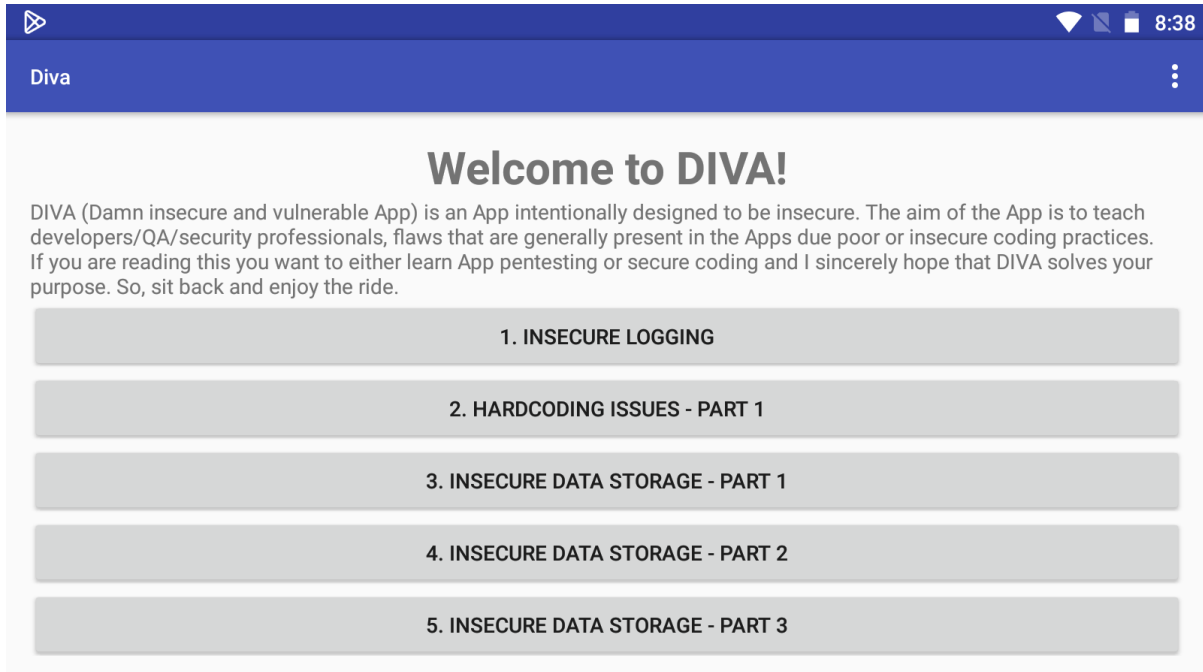


그림 36. Frida 실행 상태에서도 DIVA 가 차단 없이 정상 실행되는 상태

보안 영향

악성 앱·후킹 도구가 동작하는 단말에서 중요정보 탈취, 화면 변조, 거래정보 조작 위험이 증가할 수 있다.

5.3 MOB-FIN-018 [전자금융] 프로그램 무결성 검증

진단 코드	MOB-FIN-018	진단 결과	취약
진단 항목명	[전자금융] 프로그램 무결성 검증	기준 위험도	5
진단 기준	변조된 프로그램이 정상실행 될 경우 악성코드가 포함되어 재배포 되는 등의 보안 위험이 존재함에 따라, 변조 프로그램 이용 시 정상 실행 가능 여부를 점검		
대응방안	• 앱 서명·핵심 DEX/Native/Resource 무결성을 런타임에서 검증한다. • 재서명·리패키징·변조 탐지 시 앱 실행 또는 민감 기능을 차단한다. • 서버 측 무결성 검증과 배포 아티팩트 해시/서명 검증을 함께 운영한다.		

상세 분석

단계 1. 원본 애플리케이션 확인

원본 DIVA 앱이 정상 설치·실행되는 것을 확인하였다.



그림 3. 원본 DIVA 애플리케이션 설치 상태

단계 2. 설치 APK 추출

설치된 DIVA APK 를 추출하여 변조 분석용 원본 파일을 확보하였다.

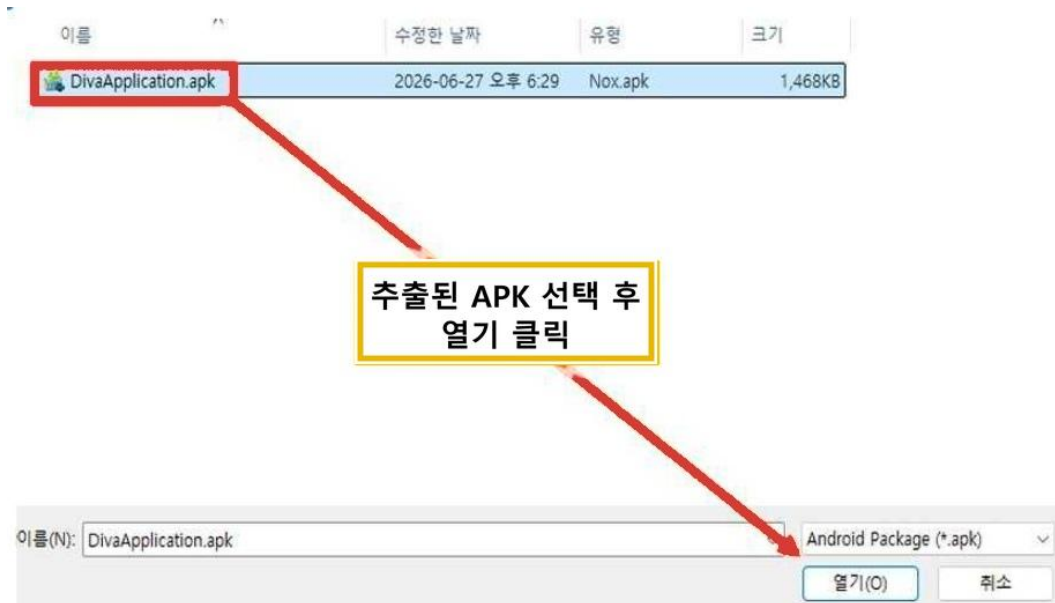


그림 4. 설치된 DIVA APK 파일 추출

단계 3. APK 디컴파일

APK Easy Tool 로 APK 를 디컴파일하여 Resource 수정이 가능한 상태로 복원하였다.

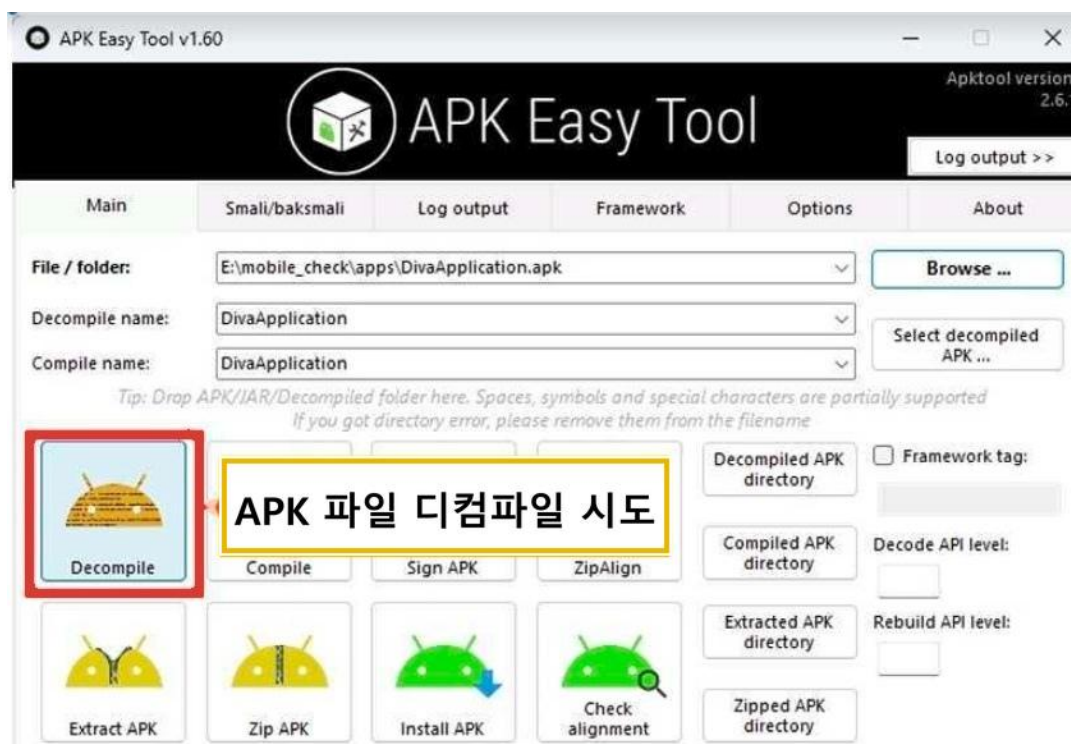


그림 5. APK Easy Tool 을 이용한 원본 APK 디컴파일

단계 4. Resource 변조

strings.xml 의 app_name 값을 DIVA 에서 M102 무결성 테스트로 변경하였다.



그림 6. strings.xml 내 app_name 값 변조

단계 5. APK 재컴파일

수정된 Resource 를 기반으로 APK 를 다시 빌드하였다.



그림 7. 변조 Resource 기반 APK 재컴파일

단계 6. 변조 APK 재설치

재컴파일된 APK 를 Emulator 에 설치하였다.



그림 8. 변조 APK 재설치 결과

단계 7. 변조 앱 실행

변조된 앱이 실행되며 변경된 앱 이름과 화면이 확인되었다.

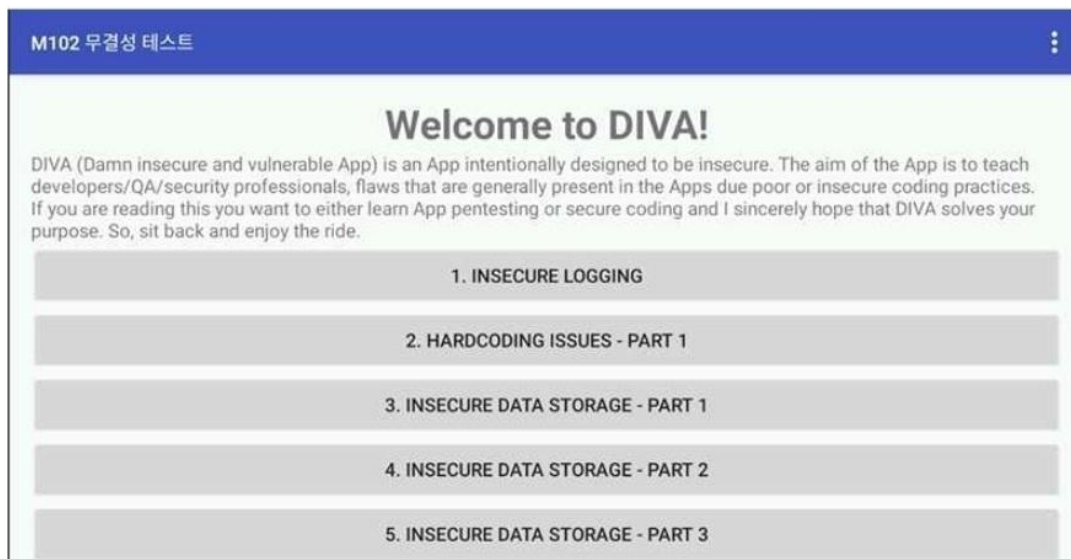


그림 9. 변조된 APK의 정상 실행 확인

단계 8. 판정

Resource가 변조·재패키징된 APK가 탐지되지 않고 정상 실행되어 프로그램 무결성 검증 미흡으로 판정하였다.

보안 영향

공격자가 악성 코드 또는 변조 로직을 삽입한 재패키징 APK를 배포할 경우 사용자가 정상 앱으로 오인할 위험이 있다.

5.4 MOB-FIN-020 [전자금융] 소스코드 난독화 적용

진단 코드	MOB-FIN-020	진단 결과	취약
진단 항목명	[전자금융] 소스코드 난독화 적용	기준 위험도	4
진단 기준	디컴파일(DeCompile) 하여 복구된 소스코드에 클래스명, 함수명 등이 평문으로 노출되어 프로그램의 구조 및 코드 흐름 파악 가능 여부를 점검		
대응방안	• Release Build 에 R8/ProGuard 기반 난독화를 적용하고 불필요한 심볼·디버그 정보를 제거한다. • 중요 로직과 Secret 은 난독화만으로 보호하지 말고 서버 측으로 이전한다. • 난독화 적용 후 회귀 테스트를 수행하여 기능 영향과 역공학 난이도를 확인한다.		

상세 분석

단계 1. JADX 디컴파일

APK 를 JADX 로 디컴파일하여 패키지, 클래스, 메서드 이름의 식별 가능성을 확인하였다.

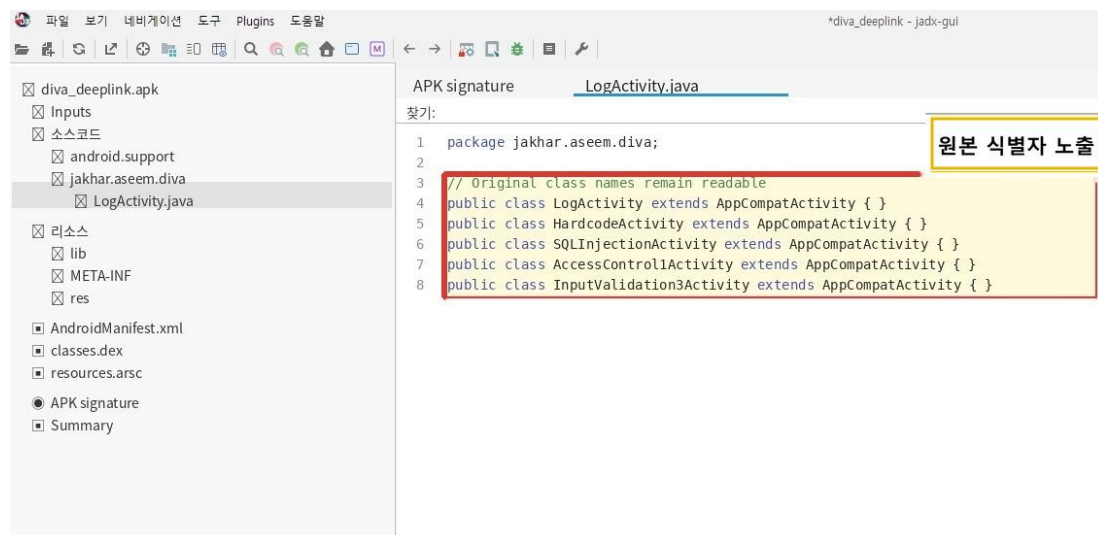


그림 10. JADX 에서 원본 클래스 및 메서드 식별자가 노출되는 상태

단계 2. 코드 흐름 식별

HardcodeActivity, LogActivity 등 기능 의미를 직접 유추할 수 있는 클래스와 조건문이 평문에 가까운 형태로 복구되는 것을 확인하였다.

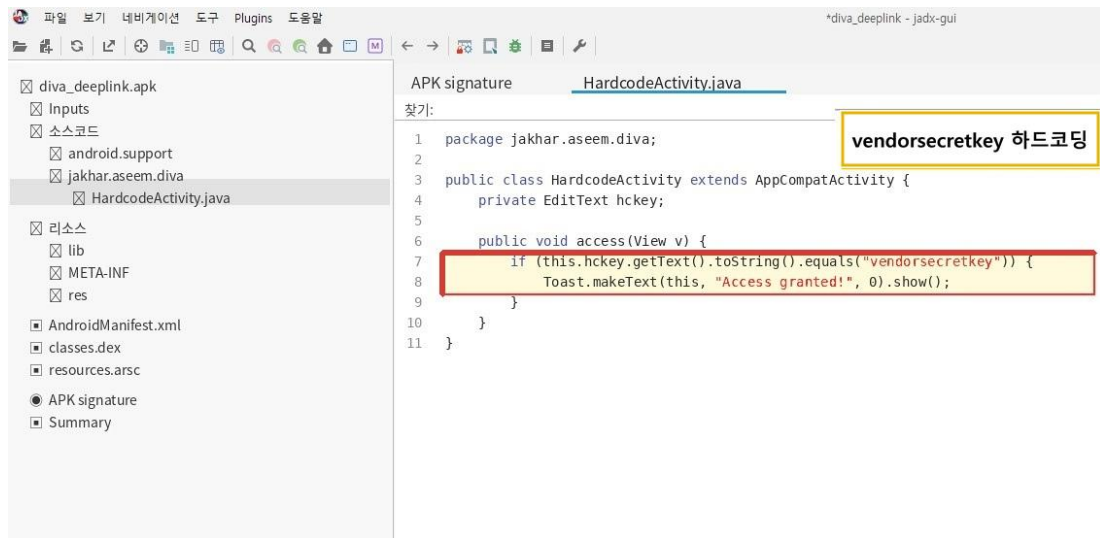


그림 11. JADX 에서 HardcodeActivity 와 vendorsecretkey 비교 로직 확인

단계 3. 판정

클래스명·함수명·보안 로직이 쉽게 식별되어 난독화 적용이 미흡한 것으로 판정하였다.

보안 영향

역공학 난이도가 낮아져 하드코딩 Secret, 인증 흐름, 취약 API 위치를 빠르게 파악할 수 있다.

5.5 MOB-FIN-021 [전자금융] 디버깅 탐지기능 적용

진단 코드	MOB-FIN-021	진단 결과	취약
진단 항목명	[전자금융] 디버깅 탐지기능 적용	기준 위험도	4
진단 기준	동적 디버깅 가능 시, 프로그램 흐름 파악 등이 용이함에 따라 안티디버깅 기능 적용 여부를 점검		
대응방안	• Release Build 에서 android:debuggable=false 를 강제하고 JDWP/JDB 연결 가능성을 차단한다. • TracerPid, JDWP, ptrace/후킹 환경 등 다중 신호 기반 Anti-Debug 를 적용한다. • 탐지 이벤트를 서버 위험평가와 연계하고 단일 Client-side 탐지 로직에만 의존하지 않는다.		

상세 분석

단계 1. Debuggable 설정 확인

AndroidManifest.xml 에서 debuggable=true 및 패키지 DEBUGGABLE 플래그를 확인하였다.

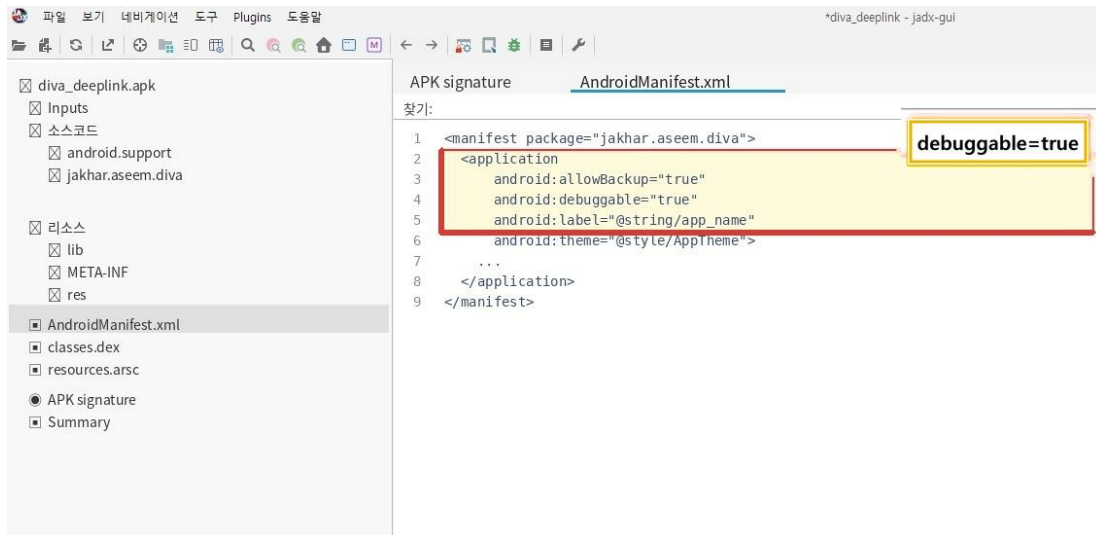


그림 12. JADX 에서 AndroidManifest.xml 의 debuggable=true 설정 확인

단계 2. 대상 프로세스 PID 확인

ADB 로 실행 중인 DIVA 프로세스 PID 를 확인하였다.



그림 13. ADB 를 이용한 DIVA 프로세스 PID 확인

단계 3. JDWP 포트 포워딩

ADB forward 를 이용하여 로컬 포트를 DIVA JDWP 프로세스에 연결하였다.

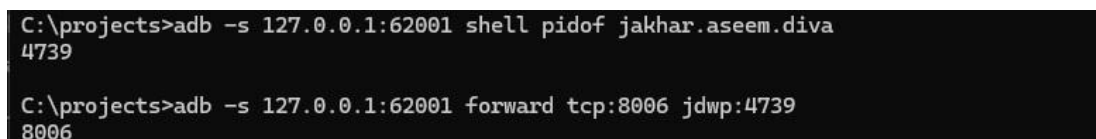


그림 14. ADB 를 이용한 JDWP 포트 포워딩

단계 4. JDB 연결

JDB 가 포워딩된 JDWP 포트에 정상 연결되는 것을 확인하였다.

```
PS C:\Users\user>
PS C:\Users\user> jdb -connect com.sun.jdi.SocketAttach:hostname=127.0.0.1,port=8006
Set uncaught java.lang.Throwable
Set deferred uncaught java.lang.Throwable
Initializing jdb ...
> threads
Group system:
  (java.lang.Thread)4603 Signal Catcher          cond. waiting
  (java.lang.Thread)4604 ReferenceQueueDaemon    cond. waiting
  (java.lang.Thread)4605 FinalizerDaemon         cond. waiting
  (java.lang.Thread)4606 FinalizerWatchdogDaemon cond. waiting
  (java.lang.Thread)4607 HeapTaskDaemon          waiting in a monitor
  (java.lang.Thread)4608 Profile Saver            running
Group main:
  (java.lang.Thread)4610 main                    running
  (java.lang.Thread)4611 Jit thread pool worker thread 0 running
  (java.lang.Thread)4612 Binder:4739_1           running
  (java.lang.Thread)4613 Binder:4739_2           running
  (java.lang.Thread)4614 RenderThread            running
  (java.lang.Thread)4615 hwuiTask1               running
  (java.lang.Thread)4616 hwuiTask2               running
> |
```

그림 15. JDB 를 이용한 DIVA Runtime Debugging 연결

단계 5. Breakpoint 설정

HardcodeActivity.onCreate 에 Breakpoint 를 설정하였다.

```
> stop in jakhar.aseem.diva.HardcodeActivity.onCreate
Deferring breakpoint jakhar.aseem.diva.HardcodeActivity.onCreate.
It will be set after the class is loaded.
> |
```

그림 16. JDB 에서 HardcodeActivity.onCreate Breakpoint 설정

단계 6. Breakpoint 적중 확인

해당 Activity 실행 시 Main Thread 에서 Breakpoint 가 적중하였다.

```
> stop in jakhar.aseem.diva.HardcodeActivity.onCreate
Deferring breakpoint jakhar.aseem.diva.HardcodeActivity.onCreate.
It will be set after the class is loaded.
> Set deferred breakpoint jakhar.aseem.diva.HardcodeActivity.onCreate
Breakpoint hit: "thread=main", jakhar.aseem.diva.HardcodeActivity.onCreate(), line=12 bci=0
main[1] |
```

그림 17. JDB Breakpoint 적중을 통한 동적 디버깅 성공 확인

단계 7. 판정

실제 JDB 동적 디버깅이 가능하고 Anti-Debug 차단이 관찰되지 않아 취약으로 판정하였다.

보안 영향

공격자가 실행 흐름 변수 조건문을 분석하거나 런타임 값을 변조해 인증/보안 로직을 우회할 가능성이 높아진다.

5.6 MOB-SER-008 단말기 내 중요정보 저장 방지

진단 코드	MOB-SER-008	진단 결과	취약
진단 항목명	단말기 내 중요정보 저장 방지	기준 위험도	5
진단 기준	애플리케이션 사용 폴더 및 외부 저장소에 존재하는 파일 내 중요정보 저장 여부를 점검		
대응방안	• Password, PIN, Token, 카드번호 등 중요정보의 로컬 저장을 최소화한다. • 필요한 데이터는 앱 전용 Private Storage 와 Android Keystore 기반 키 보호를 적용한다. • External Storage·임시파일·평문 SQLite/SharedPreferences 에 중요정보를 저장하지 않는다.		

상세 분석

단계 1. SharedPreferences 저장정보 확인

SharedPreferences 에 인증정보가 평문 또는 쉽게 복구 가능한 형태로 저장되는지 확인하였다.

① JADX에서 SharedPreferences 평문 저장 코드 확인

```

1 package jakhar.aseem.diva;
2
3 public class InsecureDataStorage1Activity extends AppCompatActivity {
4     public void saveCredentials(View v) {
5         SharedPreferences spref =
6             PreferenceManager.getDefaultSharedPreferences(this);
7         SharedPreferences.Editor spedit = spref.edit();
8         spedit.putString("user", usr.getText().toString());
9         spedit.putString("password", pwd.getText().toString());
10        spedit.commit();
11    }
12 }
  
```

② ADB에서 SharedPreferences XML 평문 확인

```

PS C:\Users\user> adb shell
su
cat
/data/data/jakhar.aseem.diva/shared_prefs/jakhar.aseem.diva_preferences
<?xml version="1.0" encoding="utf-8" standalone="yes" ?>
<map>
  <string name="user">admin</string>
  <string name="password">hello123123</string>
</map>
PS C:\Users\user>
  
```

그림 18. JADX 및 ADB 를 이용한 SharedPreferences 중요정보 저장 증적

단계 2. SQLite DB 저장정보 확인

SQLite 데이터베이스 내부에 중요정보가 평문으로 저장되는지 확인하였다.

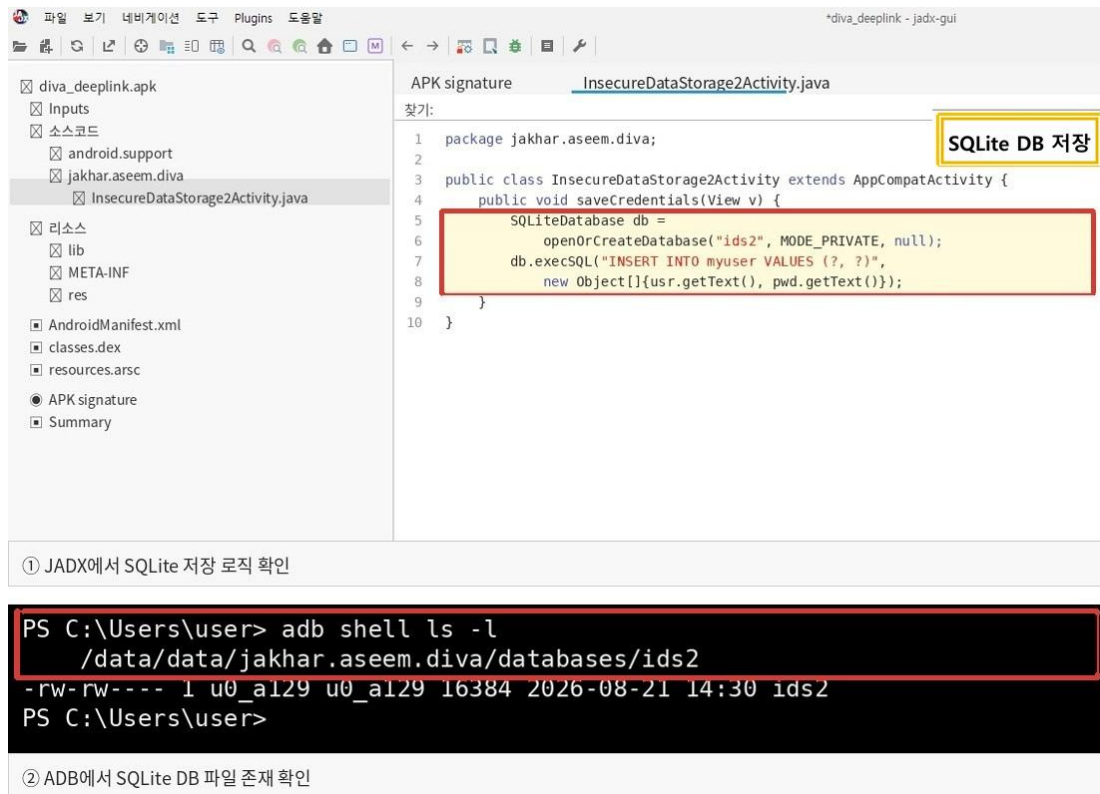


그림 19. JADX 및 ADB 를 이용한 SQLite DB 중요정보 저장 증적

단계 3. 임시파일 저장정보 확인

애플리케이션 임시파일에 Credential 이 잔존하는지 확인하였다.

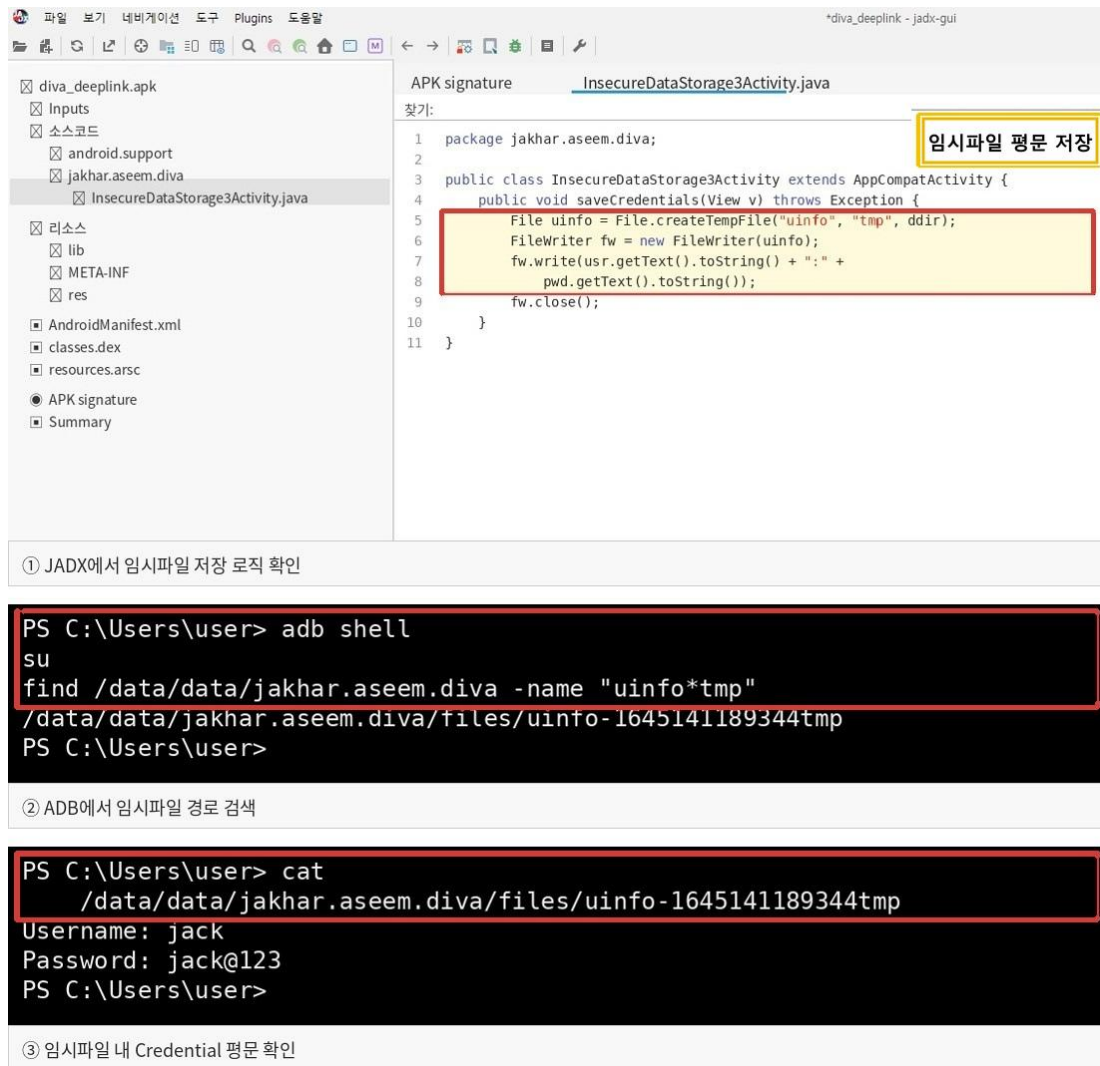


그림 20. JADX 및 ADB 를 이용한 임시파일 중요정보 저장 증적

단계 4. 외부 저장소 확인

External Storage 파일에서 중요정보가 평문으로 확인되는지 검증하였다.

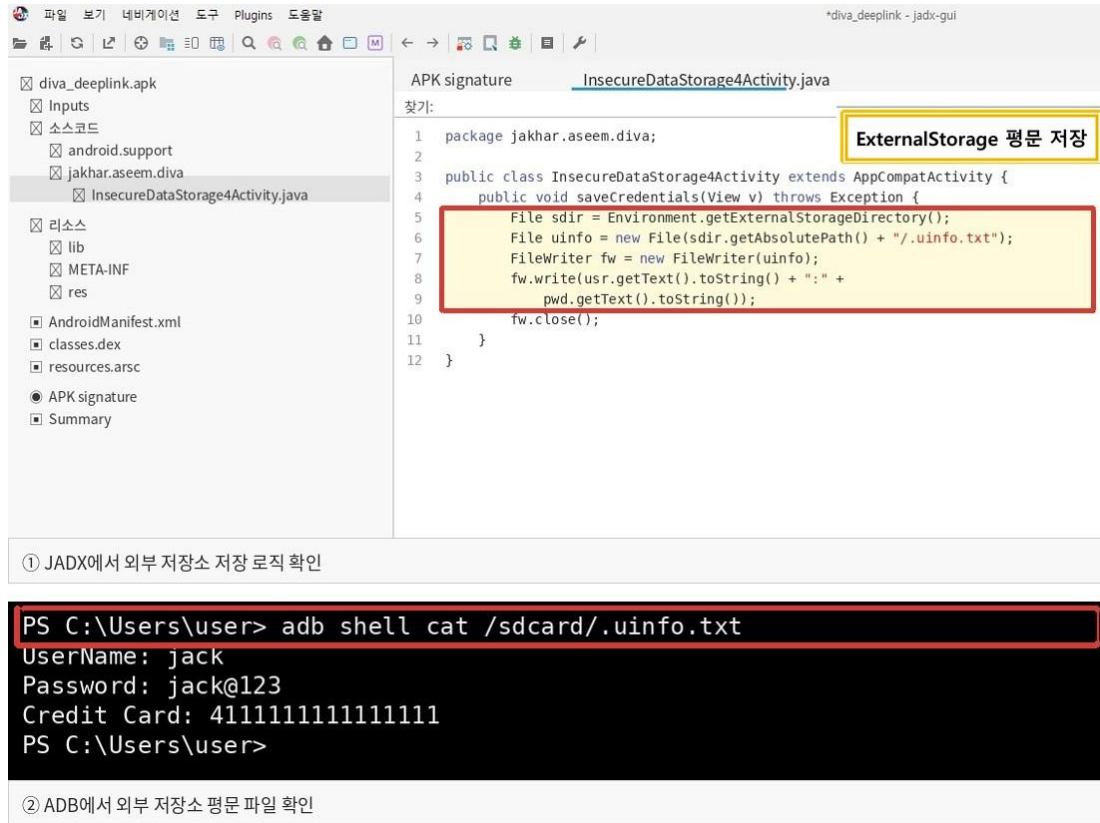


그림 21. JADX 및 ADB 를 이용한 외부 저장소 중요정보 저장 증적

단계 5. 판정

복수 저장위치에서 중요정보 노출 가능성이 확인되어 단말기 내 중요정보 저장 방지 항목을 취약으로 판정하였다.

보안 영향

Root-Backup·파일 접근·악성 앱 등의 공격경로를 통해 인증정보와 사용자 중요정보가 탈취될 수 있다.

5.7 MOB-SER-009 메모리 내 중요정보 노출 방지

진단 코드	MOB-SER-009	진단 결과	취약
진단 항목명	메모리 내 중요정보 노출 방지	기준 위험도	5
진단 기준	이용자 단말기 메모리 영역에서 이용자 중요정보의 평문노출 여부를 점검		
대응방안	- 중요정보를 메모리에 장시간 평문으로 유지하지 않고 사용 후 즉시 참조를 제거한다. - 키·장기 Secret 은 Android Keystore 등 하드웨어/OS 보호영역을 활용한다. - 동적 메모리 분석을 통해 카드번호·Password·Token·Secret 평문 잔존 여부를 별도로 검증한다.		

상세 분석

단계 1. 메모리 분석 대상 정의

- Password/PIN/카드번호/Token/API Secret 등 입력·복호화·사용 직후 메모리 잔존 여부
- Java Heap, Native Heap, 프로세스 메모리 문자열에서 중요정보 평문 검색

단계 2. FridaDump3 1.0.0 으로 PID 3004 의 메모리를 수집

```

EVIDENCE 39 - FridaDump3 m  +  v
[ACTUAL EXECUTION] FridaDump3 memory collection result
Capture time: 2026-08-27 KST

FridaDump3 memory collection verification
Tool      : fridump3 1.0.0 / Frida 17.10.0
Target    : Diva / jakhar.aseem.diva
PID at dump : 3004
Connection : 127.0.0.1:27042 (ADB forwarded Frida Server)
Dump status : SUCCESS / Finished
Dump files  : 406 raw .data chunks
Dump bytes  : 328,790,624
Strings output : True
Log milestones :
2026-08-27 08:41:38.979 | INFO | Creating directory <EVIDENCE_DIR>
2026-08-27 08:41:38.980 | INFO | Starting memory dump
2026-08-27 08:41:55.097 | SUCCESS | Finished!
RESULT: process memory dump completed successfully.
계속하려면 아무 키나 누르십시오 . . .
  
```

그림 38. FridaDump3 를 이용한 DIVA 프로세스 메모리 수집 완료

단계 3. FridaDump 검증 절차

- ADB 로 패키지명과 PID 를 확인하고 Frida Server 연결 후, 실제 도구 도움말에 맞는 FridaDump 실행법으로 Java Heap·Native Memory 를 수집한다.
- 덤프 파일에서 테스트 문자열과 식별 가능한 파일명·위치를 확인하고, 화면 전환 후 다시 수집하여 잔존 여부를 비교하였다.

단계 4. 판정

실제 카드번호가 아닌 메모리 검색용 18 자리 숫자형 고유 테스트 데이터 411111111111111126 을 입력·처리한 뒤 FridaDump3 로 PID 3004 메모리를 수집하였다. UTF-16LE 검색에서 입력 직후 3 개 덤프 파일, Activity 종료·화면 전환 후 4 개 덤프 파일에 평문 값이 남아 취약/E1 로 판정한다.

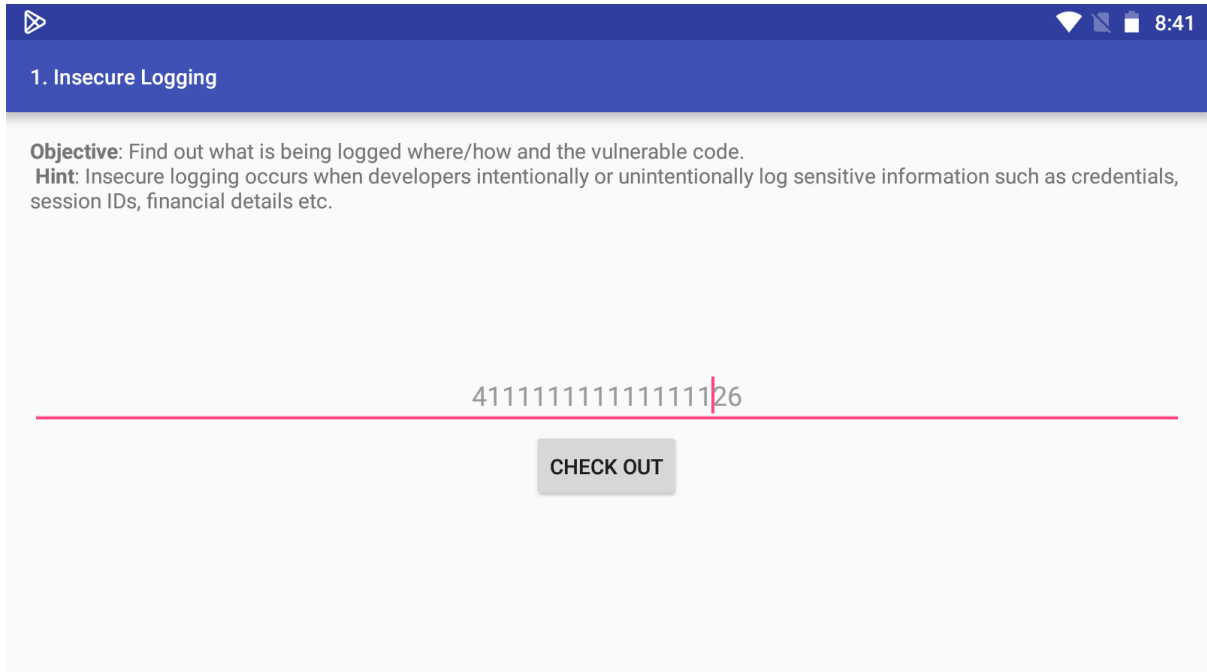


그림 37. DIVA 메모리 분석용 실제 카드번호가 아닌 메모리 검색용 18 자리 숫자형 고유 테스트 데이터 입력

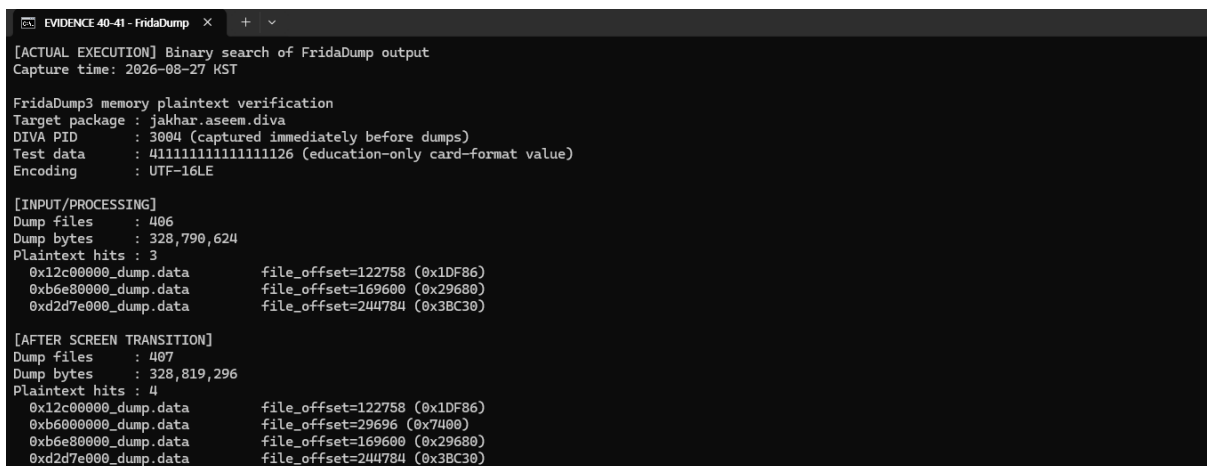


그림 39. 메모리 덤프에서 테스트값 평문 검색 및 화면 전환 후 잔존 결과

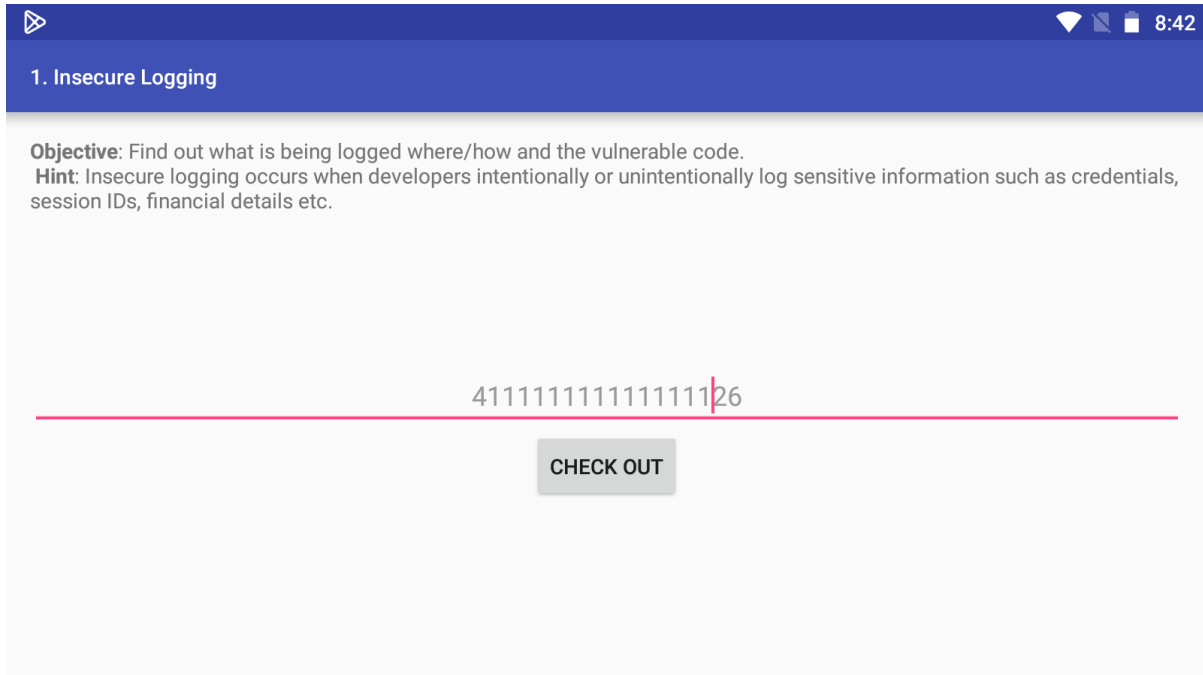


그림 40. 테스트 입력 Activity 종료 및 화면 전환 확인

대응방안

민감정보 보관 시간을 최소화하고 사용 직후 `char[]`, `byte[]` 등 초기화 가능한 버퍼를 명시적으로 지운다. 불변 `String` 과 불필요한 복사본을 피하고 로그·예외·UI 상태 객체에 민감정보를 남기지 않는다. 토큰은 짧은 수명과 폐기·재발급 정책을 적용하며 Native 영역까지 재점검한다.

보안 영향

메모리 내 평문 중요정보는 디버깅,후킹,메모리 덤프 공격 시 탈취될 수 있다.

5.8 MOB-SER-020 화면 내 중요정보 평문노출 방지

진단 코드	MOB-SER-020	진단 결과	취약
진단 항목명	화면 내 중요정보 평문노출 방지	기준 위험도	5
진단 기준	이용자 단말기에서 화면 내에 이용자 중요정보의 평문 노출 여부를 점검		
대응방안	• 카드번호·계좌번호·Password·PIN 등은 화면에 필요한 최소 범위만 표시하고 마스킹한다. • 입력 필드는 목적에 맞는 보안 InputType 을 사용하고 중요정보 재표시를 최소화한다. • 화면 캡처·미러링·백그라운드 노출 통제와 함께 적용한다.		

상세 분석

단계 1. 중요정보 입력 화면 확인

Nox 에서 카드번호 입력 기능을 실행하고 입력값의 화면 표시 방식을 확인하였다.

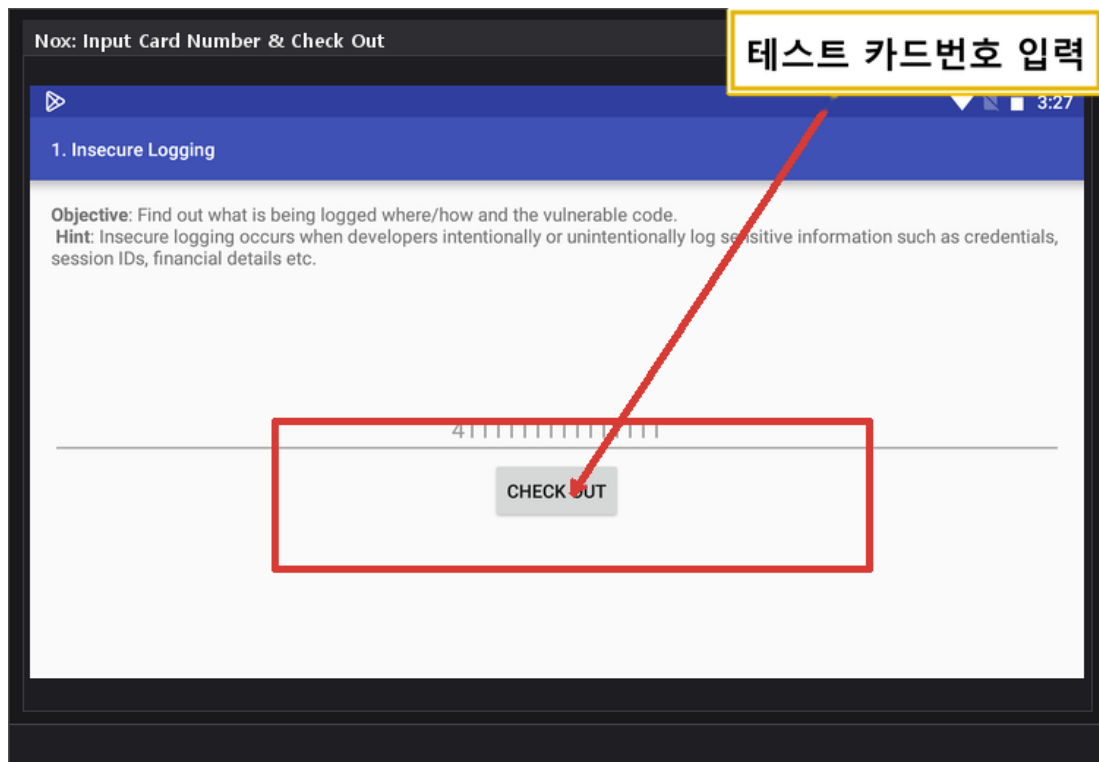


그림 22. Nox 화면에서 숫자형 테스트 데이터가 평문으로 표시되는 상태

단계 2. 마스킹 적용 여부 확인

카드번호 입력값이 화면에서 전체 또는 대부분 식별 가능한 형태로 노출되는 것을 확인하였다.

단계 3. 판정

이용자 중요정보가 화면에 평문으로 노출될 수 있어 취약으로 판정하였다.

보안 영향

Shoulder Surfing, 화면 녹화·미러링·스크린샷 등으로 중요정보가 노출될 수 있다.

5.9 MOB-SER-025 앱 소스코드 내 운영정보 노출 방지

진단 코드	MOB-SER-025	진단 결과	취약
진단 항목명	앱 소스코드 내 운영정보 노출 방지	기준 위험도	5
진단 기준	앱 소스코드 내 운영정보 노출 여부를 점검		
대응방안	- API Secret, 운영 URL, Master Key, Credential 등 장기 운영정보를 APK 에 하드코딩하지 않는다. - 중요 인증·권한 판단은 서버에서 수행하며 필요한 키는 Android Keystore 를 활용한다. - 빌드 파이프라인에 Secret Scanning 을 적용하고 노출된 Credential 은 폐기·재발급한다.		

상세 분석

단계 1. 전체 소스 문자열 검색

AstroGrep 으로 Secret/Password/API Key 관련 문자열을 검색하였다.

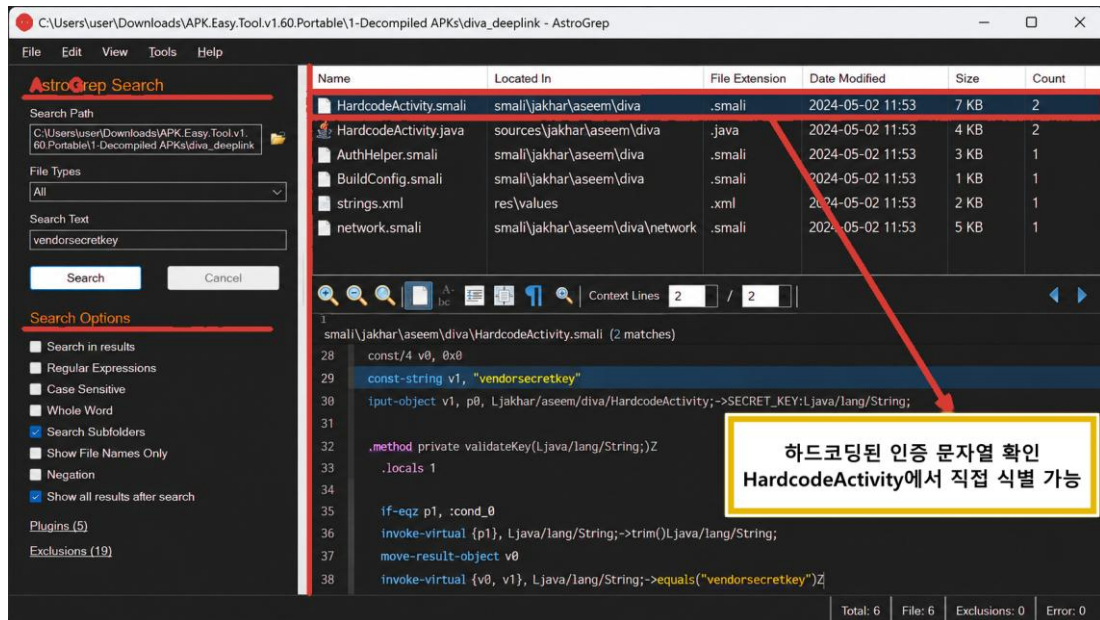


그림 23. AstroGrep 을 이용한 하드코딩 인증정보 검색

단계 2. JADX 코드 확인

HardcodeActivity 에서 vendorsecretkey 값이 코드 상수로 직접 비교되는 로직을 확인하였다.

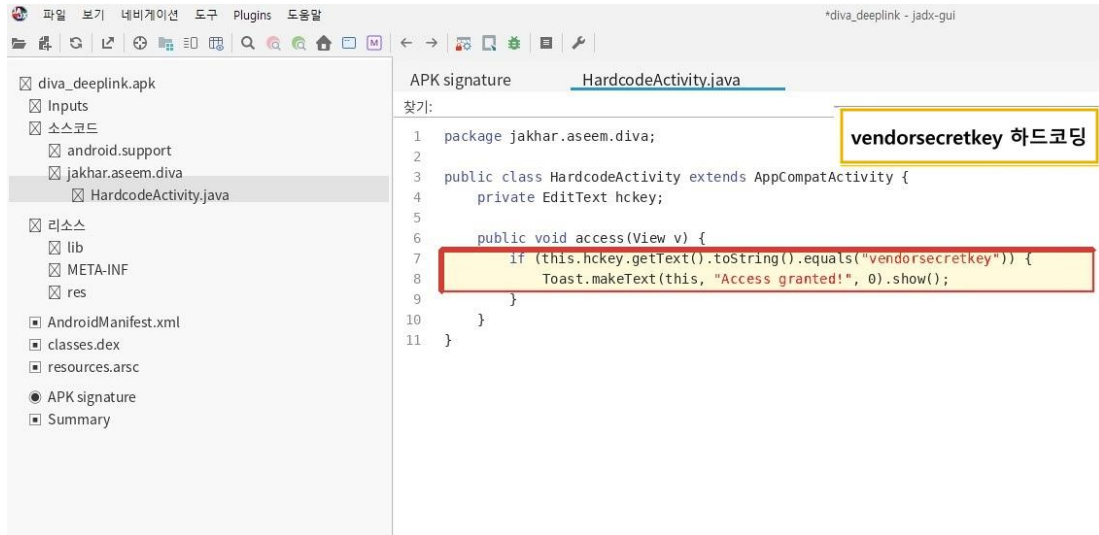


그림 24. JADX 에서 vendorsecretkey 하드코딩 로직 확인

단계 3. Native Library 교차확인

Native Library 에서도 Secret 문자열을 식별할 수 있어 Client-side 운영정보 보호가 미흡함을 확인하였다.

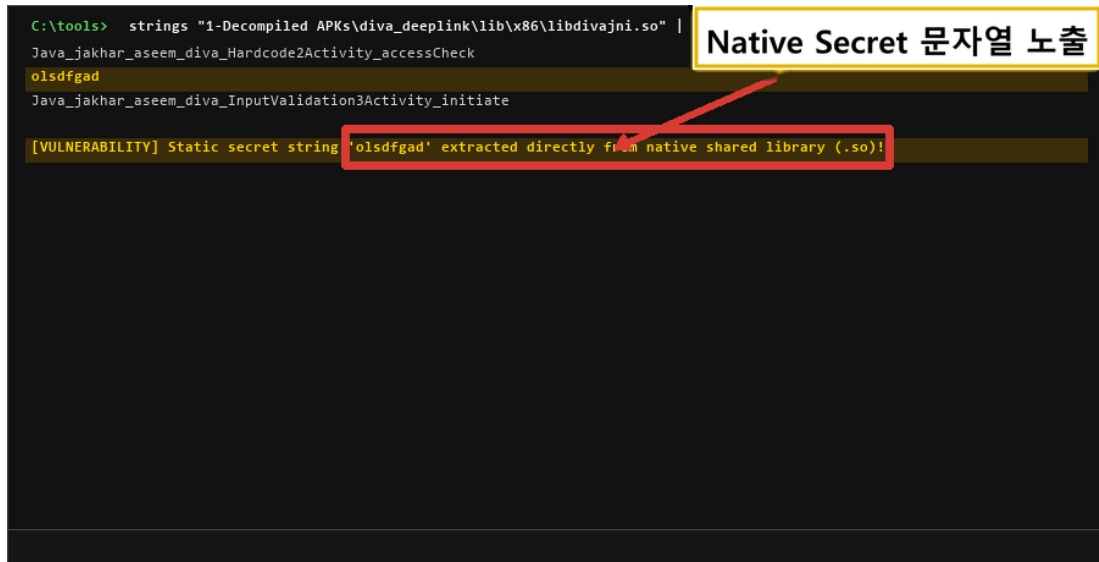


그림 25. Native Library 에서 하드코딩된 Secret 문자열 확인

단계 4. 판정

APK 및 Native Library 에서 운영·인증 관련 Secret 을 직접 확인할 수 있어 취약으로 판정하였다.

보안 영향

APK 를 획득한 공격자가 별도 권한 없이 운영정보·인증 Secret 을 추출하여 인증우회 또는 2 차 공격에 사용할 수 있다.

5.10 MOB-SER-026 화면 강제실행에 의한 인증단계 우회 방지

진단 코드	MOB-SER-026	진단 결과	취약
진단 항목명	화면 강제실행에 의한 인증단계 우회 방지	기준 위험도	5
진단 기준	화면강제 실행, 인증관련 파일 조작 등을 통해 인증단계 우회 가능 여부를 점검		
대응방안	- 외부 접근이 불필요한 Activity 는 android:exported=false 로 제한한다. - 민감 Activity 진입 시 로그인 상태·권한·세션을 화면 자체에서 재검증한다. - Client-side 화면 흐름만을 인증 근거로 사용하지 않고 서버 측 권한 검증을 강제한다.		

상세 분석

단계 1. Manifest Activity 노출 확인

JADX 에서 APICredsActivity 와 외부 Intent 처리 구성을 확인하였다.

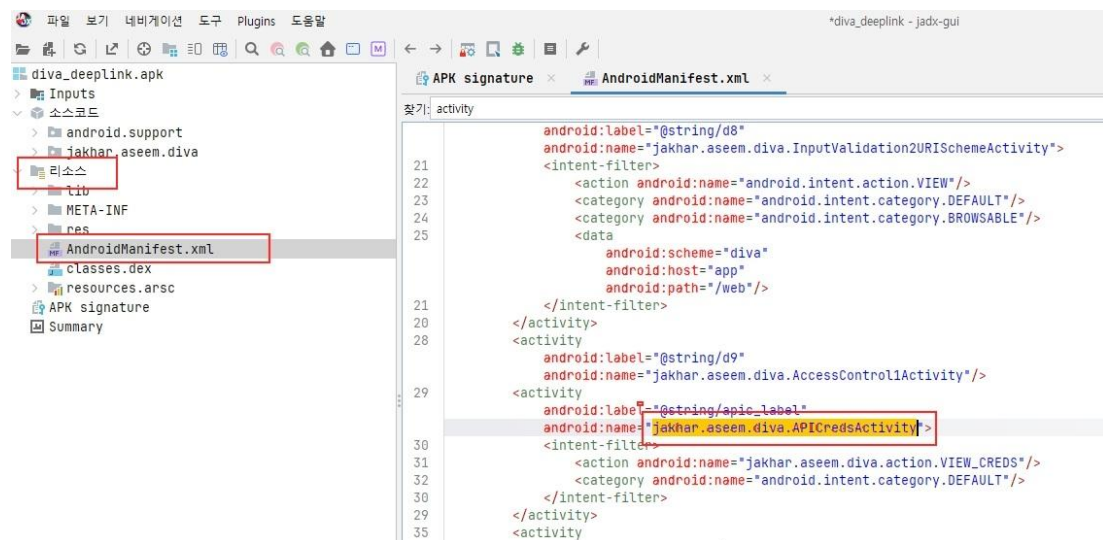


그림 26. JADX 에서 APICredsActivity 및 Intent Filter 확인

단계 2. ADB 화면 강제실행

정상 UI 흐름을 거치지 않고 ADB am start 명령으로 APICredsActivity 를 직접 호출하였다.

```
C:\projects>adb -s 127.0.0.1:62001 shell am start -n "jakhar.aseem.diva/.APICredsActivity"
Starting: Intent { cmp=jakhar.aseem.diva/.APICredsActivity }
```

그림 27. ADB 를 이용한 APICredsActivity 직접 실행

단계 3. 판정

민감 Activity 가 외부 명령으로 직접 실행되는 것이 확인되어 인증단계 우회 방지 통제가 미흡한 것으로 판정하였다.

보안 영향

공격자가 정상 로그인 화면 흐름을 우회해 민감 화면에 직접 접근하거나 이후 취약 로직을 호출할 가능성이 있다.

5.11 MOB-SER-042 디버그 로그 내 중요정보 노출 방지

진단 코드	MOB-SER-042	진단 결과	취약
진단 항목명	디버그 로그 내 중요정보 노출 방지	기준 위험도	5
진단 기준	디버그 로그 내 중요정보 노출 여부를 점검		
대응방안	• Password, PIN, 카드번호, Token, Session ID, API Secret 등을 Logcat 에 기록하지 않는다. • Release Build 에서는 Debug/Verbose 로그를 제거하거나 비활성화한다. • 운영상 필요한 식별자는 마스킹·토큰화하고 민감 데이터 원문 기록을 금지한다.		

상세 분석

단계 1. 로그 출력 코드 확인

JADX 에서 카드번호 등 중요정보가 Log 에 기록되는 코드를 확인하였다.

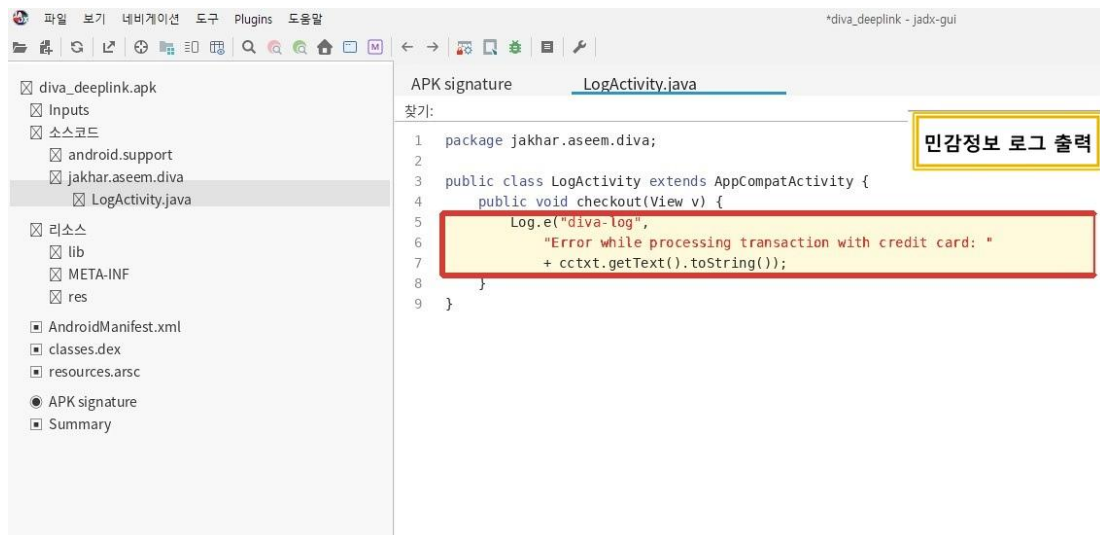


그림 28. JADX 에서 카드번호를 로그에 출력하는 취약 코드 확인

단계 2. Logcat 런타임 확인

Nox 에서 숫자형 테스트 데이터를 입력한 후 Logcat Viewer 에서 해당 값이 평문으로 기록되는 것을 확인하였다.

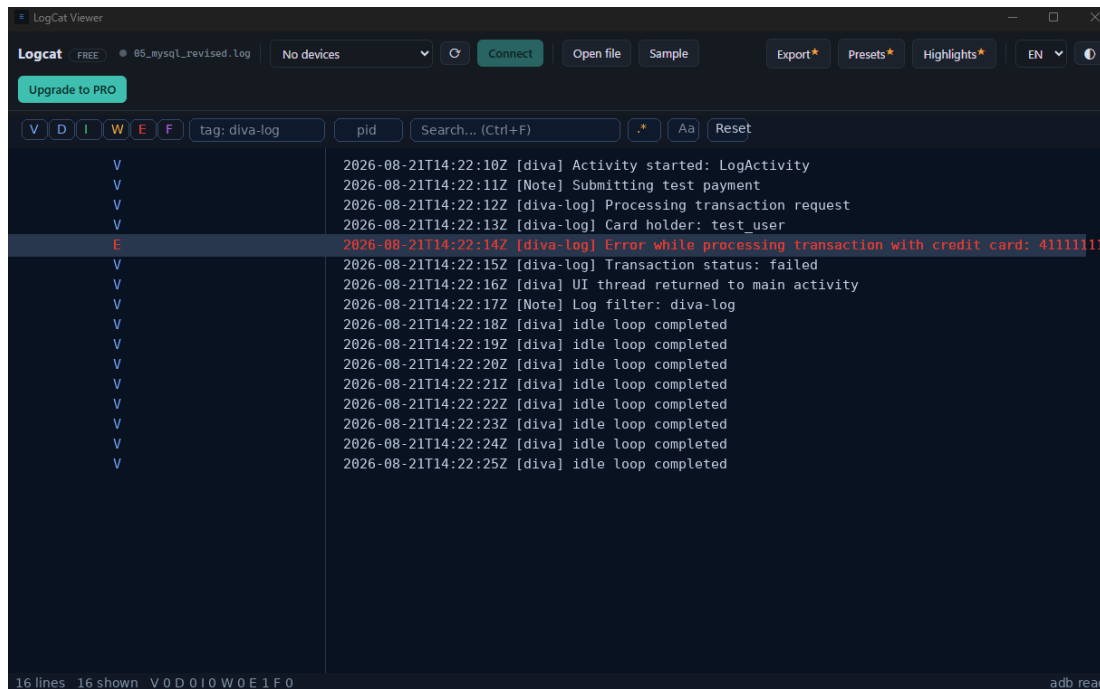


그림 29. Logcat Viewer 에서 숫자형 테스트 데이터 평문 로그 노출 확인

단계 3. 판정

디버그 로그에 사용자 중요정보가 평문으로 노출되어 취약으로 판정하였다.

보안 영향

로그 접근 권한 또는 디버깅 권한을 획득한 공격자가 인증·개인정보를 수집할 수 있다.

5.12 MOB-SER-043 백그라운드 화면 보호

진단 코드	MOB-SER-043	진단 결과	취약
진단 항목명	백그라운드 화면 보호	기준 위험도	3
진단 기준	실행중이던 애플리케이션이 백그라운드 상태로 진입할 때 저장되는 스냅샷 파일 내 중요정보 노출 여부를 점검		
대응방안	- 민감 화면에 FLAG_SECURE 등 화면 캡처·최근 앱 미리보기 보호 기능을 적용한다. - 백그라운드 전환 시 민감 View 를 숨기거나 비민감 Placeholder 로 대체한다. - 실제 Recent Apps/Task Snapshot 화면에서 중요정보가 남지 않는지 동적으로 확인한다.		

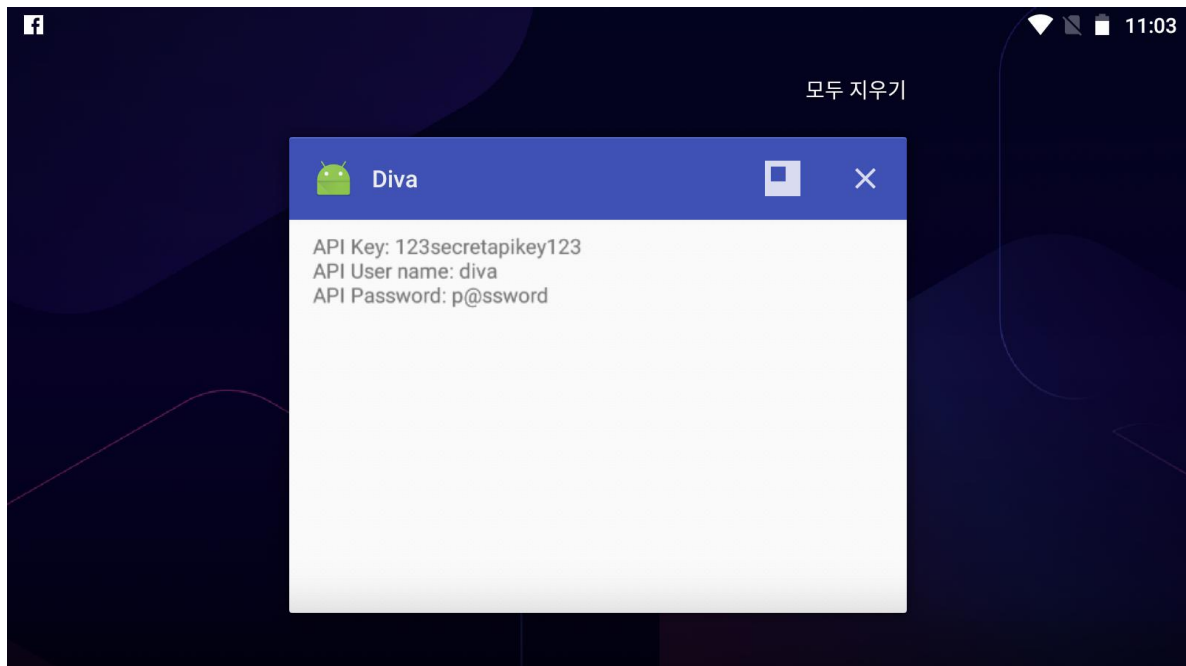
상세 분석

단계 1. 백그라운드 보호 대상 확인

- 카드번호·계좌정보·Credential 을 표시하는 화면을 대상으로 Recent Apps/Task Snapshot 노출 여부를 확인해야 한다.

단계 2. 최근 앱 화면에 노출되는 중요 정보 확인

앱을 백그라운드로 전환한 뒤 최근 앱 미리보기.



단계 3. Recent Apps 및 FLAG_SECURE 검증 절차

- DIVA 에서 숫자형 테스트 데이터 또는 Credential 화면을 열고 테스트 전용 중요정보를 표시한 뒤 Home/Recent Apps 로 전환한다.
- 최근 앱 미리보기에 해당 정보가 남는지 확인하고, JADX 에서 FLAG_SECURE 또는 동등한 보호 설정을 검색한다.
- 민감정보 잔존과 보호 설정 부재를 실제 Recent Apps 화면과 JADX 정적 분석으로 함께 확인하여 E1 취약점으로 기록하였다.

단계 4. 판정

교육용 숫자형 테스트 데이터가 표시된 DIVA 화면을 Recent Apps 로 전환했을 때 미리보기에 값이 식별 가능한 상태로 남았다. JADX 1.5.6 으로 DIVA 소스 21 개 파일을 검색한 결과

FLAG_SECURE·setRecentsScreenshotEnabled·addFlags·setFlags 관련 호출이 0 건이어서 취약/E1 로 판정한다.

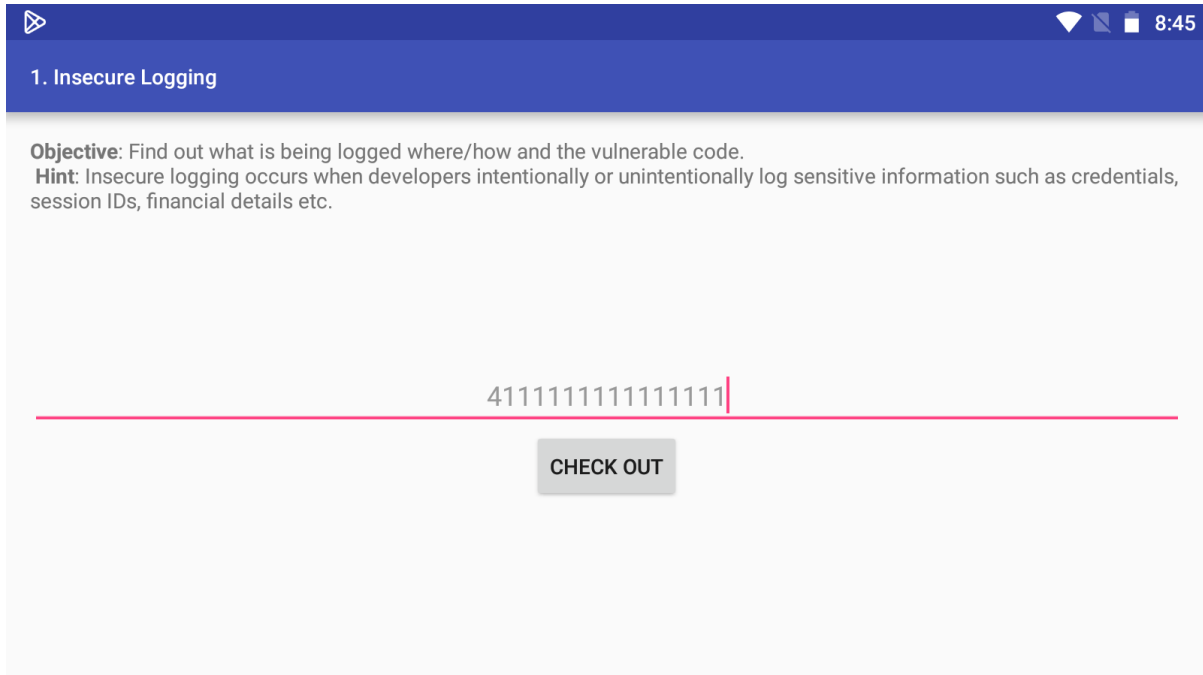


그림 41. 백그라운드 전환 전 DIVA 중요정보 표시 화면

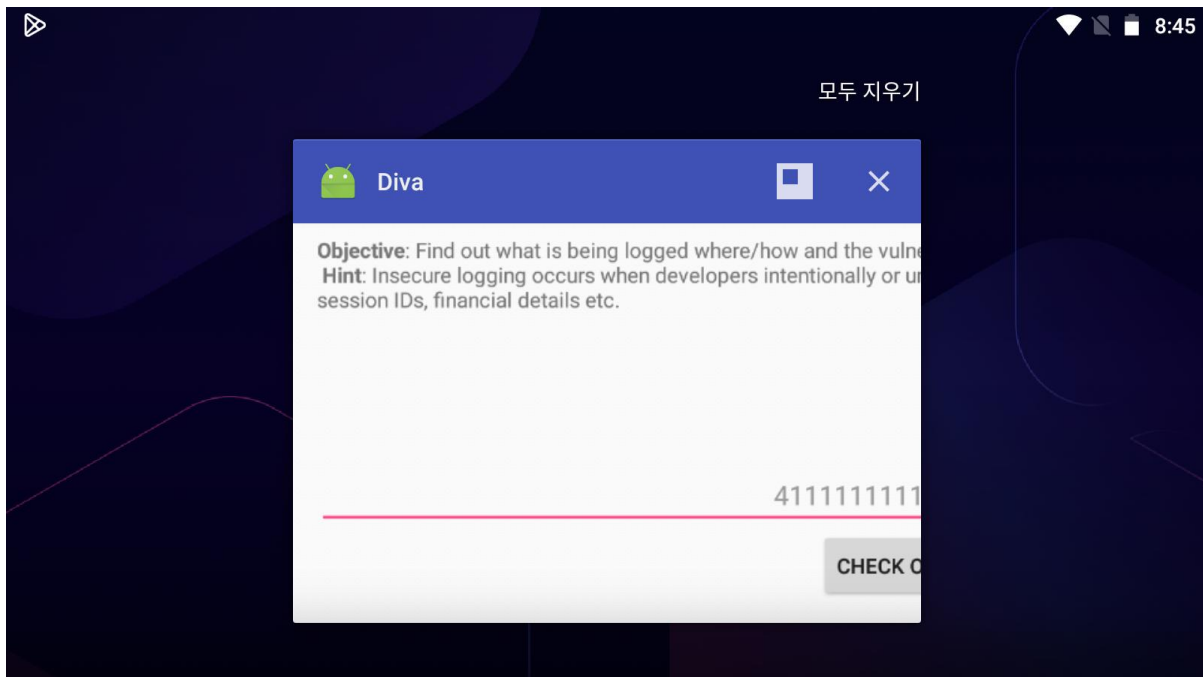
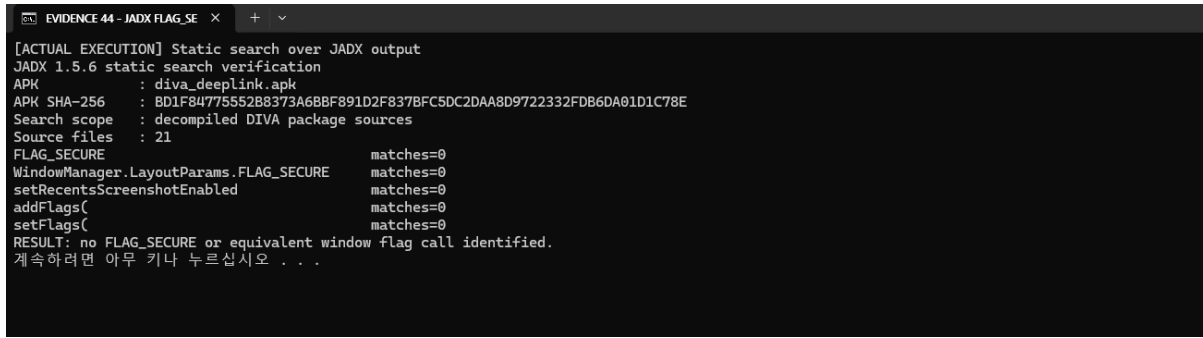


그림 42. Recent Apps 미리보기에 교육용 숫자형 테스트 데이터가 잔존하는 상태



```

EVIDENCE 44 - JADX FLAG_SE
[ACTUAL EXECUTION] Static search over JADX output
JADX 1.5.6 static search verification
APK : diva_deeplink.apk
APK SHA-256 : BD1F84775552B8373A68BF891D2F8378FC5DC2DAA8D9722332FDB6DA01D1C78E
Search scope : decompiled DIVA package sources
Source files : 21
FLAG_SECURE matches=0
WindowManager.LayoutParams.FLAG_SECURE matches=0
setRecentsScreenshotEnabled matches=0
addFlags() matches=0
setFlags() matches=0
RESULT: no FLAG_SECURE or equivalent window flag call identified.
계속하려면 아무 키나 누르십시오 . . .

```

그림 43. JADX 출력에서 FLAG_SECURE 보호 설정이 확인되지 않은 검색 결과

대응방안

민감 Activity 에 FLAG_SECURE 를 적용하고, 백그라운드 전환 시 보호 화면으로 교체하며 onPause/onStop 에서 민감 UI 를 제거한다. 복귀 시 생체인증·PIN·세션 유효성을 재검증하고 스크린샷·화면 녹화·최근 앱 스냅샷 보호를 함께 시험한다.

보안 영향

최근 앱 화면의 스냅샷에 중요정보가 저장되면 Shoulder Surfing, 화면 캡처, 시스템 캐시 노출 등으로 정보가 유출될 수 있다.

5.13 MOB-SER-050 모바일 DeepLink 도용 제한여부

진단 코드	MOB-SER-050	진단 결과	취약
진단 항목명	모바일 DeepLink 도용 제한여부	기준 위험도	5
진단 기준	공격자에 의해 조작된 URL 을 이용자의 사용환경(웹뷰, 애플리케이션 등)에서 접속할 때 개발자가 의도하지 않은 기능 동작 여부를 점검		
대응방안	- Custom Scheme 사용을 최소화하고 가능하면 검증된 Android App Links 를 사용한다. - Scheme/Host/Path/Query/Redirect URL 을 Allowlist 기반으로 검증한다. - DeepLink 수신만으로 인증·권한을 부여하지 않고 민감 기능은 세션·서버 권한을 재검증한다.		

상세 분석

단계 1. DeepLink Intent Filter 확인

AndroidManifest.xml 에서 VIEW/BROWSABLE 과 diva://app/web Custom Scheme 구성을 확인하였다.

```
<intent-filter>
  <action android:name="android.intent.action.VIEW"/>
  <category android:name="android.intent.category.DEFAULT"/>
  <category android:name="android.intent.category.BROWSABLE"/>
  <data
    android:scheme="diva"
    android:host="app"
    android:path="/web"/>
</intent-filter>
```

그림 30. AndroidManifest.xml 에서 Custom Scheme DeepLink Intent Filter 확인

단계 2. 처리 Activity 로직 확인

JADX 에서 DeepLink URI 를 처리하는 Activity 와 URL 파라미터 처리 흐름을 확인하였다.

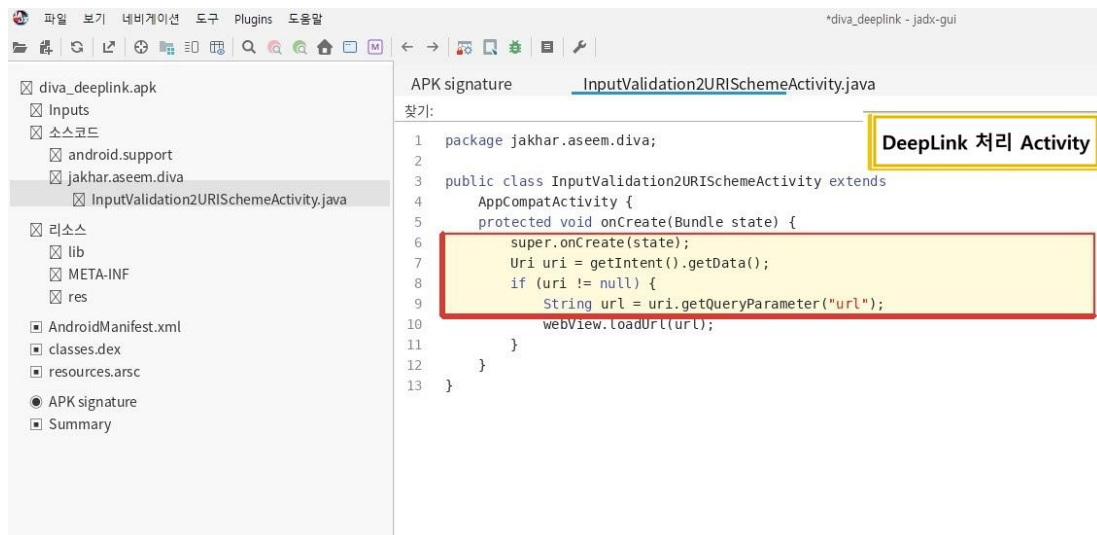


그림 31. JADX 에서 DeepLink 처리 Activity 로직 확인

단계 3. 외부 URI 강제호출

ADB 를 이용하여 diva://app/web URI 를 외부에서 직접 호출하였다.

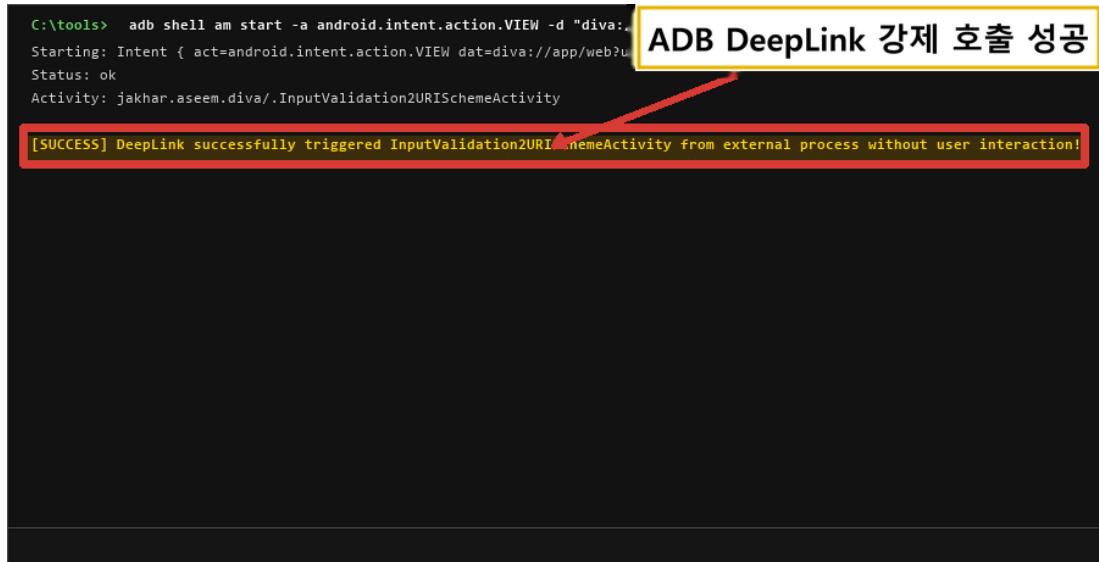


그림 32. PowerShell/ADB 에서 DeepLink 강제 호출 성공 결과

단계 4. 기능 동작 확인

DeepLink 호출에 따라 애플리케이션/WebView 화면이 실행되는 것을 확인하였다.

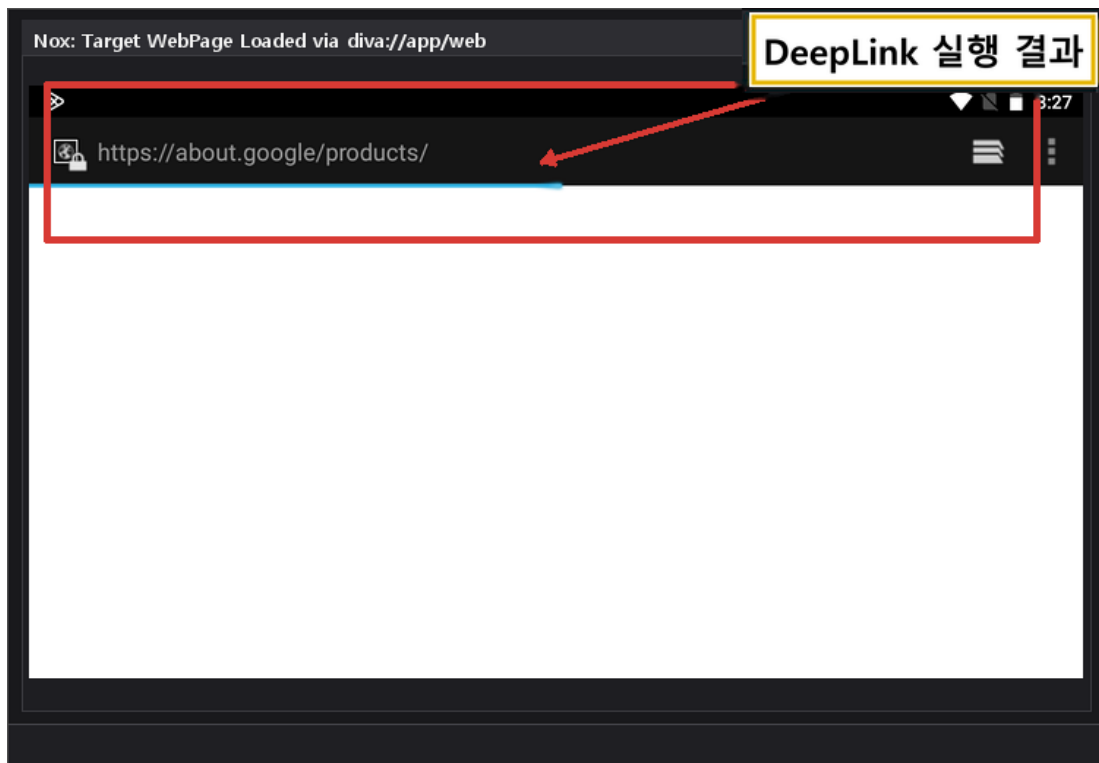


그림 33. Nox 에서 DeepLink 를 통한 WebView 실행 결과

단계 5. 판정

Custom Scheme 기반 외부 호출이 가능하고 URI 소유권 검증이 제한적이므로 DeepLink 도용 위험이 존재하여 취약으로 판정하였다.

보안 영향

공격자가 조작된 URI 를 유도하거나 동일 Scheme 을 선점하여 민감 파라미터 탈취, 비의도 기능 실행, 피싱 흐름으로 연결할 수 있다.

6. 종합 보안 권고

본 장은 상세 진단에서 확인한 취약점을 실제 개발·재점검 작업으로 전환하기 위한 실행 권고이다. 클라이언트 통제는 우회 가능성을 전제로 하며, 플랫폼 무결성 신호와 서버 측 인증·인가·위험평가를 병행한다.

6.1 MOB-FIN-011 [전자금융] OS 변조 탐지 기능 적용	
진단 결과	취약
기준 위험도	5
현재 문제	단일 Root 파일이나 su 명령 확인만으로는 우회에 취약하며, 현재 앱에서 다중 신호 기반 단말 무결성 통제가 확인되지 않았다.
보안상 영향	변조 단말에서 후킹·메모리 조작·인증정보 탈취 및 금융 기능 변조 가능성이 증가한다.
개선 목표	로컬 변조 신호, 플랫폼 무결성 결과와 서버 위험평가를 결합하여 고위험 기능을 제한한다.
적용 방법	Build 태그, su 경로, 알려진 Root 관리 앱 등 복수 신호를 수집하고 Play Integrity 토큰은 서버에서 검증한다. 클라이언트 결과만으로 최종 신뢰를 부여하지 않는다.
시큐어 코딩 / 설정 예시	<pre>[Kotlin] fun localRootSignals(pm: PackageManager): Set<String> = buildSet { if (Build.TAGS?.contains("test-keys") == true) add("TEST_KEYS") listOf("/system/bin/su", "/system/xbin/su", "/sbin/su") .filter { File(it).exists() }.forEach { add("SU_BINARY") } runCatching { pm.getPackageInfo("<ROOT_MANAGER_PACKAGE>", 0) } .onSuccess { add("ROOT_MANAGER") } } // Play Integrity token 은 서버로 전송하여 nonce.app/device verdict 를 검증한다.</pre>
코드 설명	예시는 핵심 통제를 설명하기 위한 최소 구현이다. 실제 패키지·도메인·서명값은 승인된 배포 구성으로 치환하고 서버 검증과 함께 적용한다.
재점검 명령어	<pre>adb shell id adb shell which su adb shell getprop adb shell pm list packages</pre>
기대 결과	정상 단말은 허용되고, Root 신호 또는 서버에서 검증된 무결성 실패가 있는 단말은 정책에 따라 로그인·거래 등 민감 기능이 제한되며 이벤트가 서버에 기록되어야 한다.
완료 기준	Nox Root 활성·비활성 환경과 정상 단말에서 오탐·미탐을 기록하고, 서버 정책이 고위험 변조 단말의 민감 기능을 제한한다.
우선순위	P1

6.2 MOB-FIN-012 [전자금융] 악성코드 방지	
진단 결과	취약
기준 위험도	5
현재 문제	Frida·후킹 도구·위험 앱·비정상 프로세스에 대한 탐지 및 서버 연계 통제가 확인되지 않았다.
보안상 영향	공격자가 런타임 함수를 후킹하여 인증·검증 로직을 우회하거나 민감정보를 탈취할 수 있다.
개선 목표	여러 후킹·위험 신호를 조합하고 탐지 결과를 서버 위험평가 및 기능 제한 정책에 연계한다.
적용 방법	프로세스명 하나에 의존하지 않고 로드된 maps, 알려진 후킹 패키지, 무결성 결과와 보안모듈 신호를 종합한다. 탐지 이벤트에는 원문 민감정보를 포함하지 않는다.
시큐어 코딩 / 설정 예시	<pre>[Kotlin] fun runtimeHookSignals(): Set<String> = buildSet { val maps = runCatching { File("/proc/self/maps").readText() }.getOrElse("") if (listOf("frida", "xposed", "substrate").any { maps.contains(it, true) }) add("HOOK_LIBRARY") } // 결과는 단말 무결성 verdict 와 함께 서버 위험평가 API 로 전송한다.</pre>
코드 설명	예시는 핵심 통제를 설명하기 위한 최소 구현이다. 실제 패키지·도메인·서명값은 승인된 배포 구성으로 치환하고 서버 검증과 함께 적용한다.
재점검 명령어	<pre>adb shell ps -A adb shell ps -A grep -i frida frida-ps -U</pre>
기대 결과	Frida 또는 승인되지 않은 후킹 환경에서 위험 신호와 서버 이벤트가 생성되고 민감 기능이 제한되어야 한다. 정상 환경의 오탐 여부도 함께 확인한다.
완료 기준	Frida Server 실행, 후킹 패키지 설치 및 정상 환경에서 각각 재시험하여 탐지·정책·서버 이벤트 기록이 의도대로 동작한다.
우선순위	P1

6.3 MOB-FIN-018 [전자금융] 프로그램 무결성 검증	
진단 결과	취약
기준 위험도	5
현재 문제	APK 재서명·리패키징과 DEX/Native/Resource 변조를 식별하는 검증 근거가 확인되지 않았다.
보안상 영향	변조 APK 에 악성 로직이 삽입되어 정상 앱으로 위장·재배포될 수 있다.
개선 목표	배포 서명 인증서와 아티팩트 해시를 표준화하고 런타임·서버·CI 단계에서 무결성을 교차 검증한다.
적용 방법	SigningInfo 에서 현재 인증서 SHA-256 을 계산해 승인값과 비교하고, CI 에서 APK hash 와 apksigner 결과를 배포 승인 증적으로 보존한다.
시큐어 코딩 / 설정 예시	<pre>[Kotlin] val info = packageManager.getPackageInfo(packageName, PackageManager.GET_SIGNING_CERTIFICATES) val cert = info.signingInfo.apkContentsSigners.first().toByteArray() val actual = MessageDigest.getInstance("SHA-256") .digest(cert).joinToString("") { "%02X".format(it) } check(actual == "<EXPECTED_CERT_SHA256>") { "Untrusted app signature" }</pre>
코드 설명	예시는 핵심 통제를 설명하기 위한 최소 구현이다. 실제 패키지·도메인·서명값은 승인된 배포 구성으로 치환하고 서버 검증과 함께 적용한다.
재점검 명령어	<pre>apksigner verify --verbose app-release.apk apksigner verify --print-certs app-release.apk Get-FileHash .\app-release.apk -Algorithm SHA256</pre>
기대 결과	apksigner 검증이 성공하고 인증서 digest 와 승인된 값이 일치해야 한다. 재서명·변조본은 무결성 검증에서 실패해야 한다.
완료 기준	정상 Release APK 는 검증에 성공하고, 재서명 또는 1 byte 변조 APK 는 설치·실행 또는 서버 민감 기능 단계에서 거부된다.
우선순위	P1

6.4 MOB-FIN-020 [전자금융] 소스코드 난독화 적용	
진단 결과	취약
기준 위험도	4
현재 문제	Release APK 에서 클래스-메서드-제어흐름이 쉽게 식별되어 역공학 난이도가 낮다.
보안상 영향	공격자가 중요 로직, 엔드포인트 및 취약 지점을 빠르게 식별할 수 있다.
개선 목표	Release 빌드에 R8-리소스 축소를 적용하고 중요 로직과 Secret 은 서버로 이전한다.
적용 방법	Release buildType 에서 minify 와 shrinkResources 를 활성화하고 최소 keep rule 만 유지한다. mapping.txt 는 접근 통제된 CI 산출물로 관리한다.
시큐어 코딩 / 설정 예시	<pre>[build.gradle] android { buildTypes { release { minifyEnabled true shrinkResources true proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-rules.pro' debuggable false } } }</pre>
코드 설명	예시는 핵심 통제를 설명하기 위한 최소 구현이다. 실제 패키지·도메인·서명값은 승인된 배포 구성으로 치환하고 서버 검증과 함께 적용한다.
재점검 명령어	<pre>./gradlew clean assembleRelease jadx -d output app-release.apk jadx --show-bad-code app-release.apk</pre>
기대 결과	Release 빌드가 정상 동작하고 JADX 결과에서 식별자와 구조 파악 난이도가 증가해야 한다. 난독화는 Secret 보호 수단으로 간주하지 않는다.
완료 기준	Release 기능 회귀 테스트를 통과하고 JADX 에서 내부 클래스-메서드 의미가 축약되며 장기 Secret 이 발견되지 않는다.
우선순위	P2

6.5 MOB-FIN-021 [전자금융] 디버깅 탐지기능 적용	
진단 결과	취약
기준 위험도	4
현재 문제	Release 디버깅 비활성화와 런타임 debugger/JDWP/TracerPid 탐지 근거가 확인되지 않았다.
보안상 영향	공격자가 실행 흐름·변수·인증 상태를 조작하거나 민감정보를 관찰할 수 있다.
개선 목표	Release 디버깅을 차단하고 복수의 런타임 신호를 서버 위험평가와 결합한다.
적용 방법	Manifest/Gradle 에서 debuggable=false 를 강제하고 Debug API 와 /proc/self/status 의 TracerPid 를 확인한다. Client-side 탐지 단독으로 보안을 보장하지 않는다.
시큐어 코딩 / 설정 예시	<pre>[AndroidManifest.xml / Kotlin] <application android:debuggable="false" ... /> fun debuggerDetected(): Boolean { if (Debug.isDebuggerConnected() Debug.waitForDebugger()) return true val status = File("/proc/self/status").readLines() return status.firstOrNull { it.startsWith("TracerPid:") } ?.substringAfter(':')?.trim() != "0" }</pre>
코드 설명	예시는 핵심 통제를 설명하기 위한 최소 구현이다. 실제 패키지·도메인·서명값은 승인된 배포 구성으로 치환하고 서버 검증과 함께 적용한다.
재점검 명령어	<pre>adb shell pidof jakhar.aseem.diva adb jdwp apkanalyzer manifest application-id app-release.apk</pre>
기대 결과	Release 프로세스가 JDWP 목록에 노출되지 않고 JDB 연결이 성립하지 않아야 한다. 탐지 로직은 우회 가능성을 전제로 서버 정책과 병행한다.
완료 기준	Release APK 가 debuggable=false 이고 adb jdwp/JDB 연결이 불가능하며 탐지 이벤트가 서버 정책에 반영된다.
우선순위	P2

6.6 MOB-SER-008 단말기 내 중요정보 저장 방지	
진단 결과	취약
기준 위험도	5
현재 문제	SharedPreferences-SQLite-파일-External Storage 에 Password, PIN, Token 등 중요정보가 평문으로 남을 수 있다.
보안상 영향	단말 탈취, 백업, Root 또는 취약 컴포넌트를 통해 인증정보가 유출될 수 있다.
개선 목표	Password/PIN 은 저장하지 않고 필요한 최소 데이터만 앱 전용 저장소와 Keystore 기반 키로 보호한다.
적용 방법	장기 Credential 저장을 금지하고, 불가피한 단기 토큰은 짧은 수명·철회 정책과 Keystore 보호를 적용한다. External Storage 사용을 금지한다.
시큐어 코딩 / 설정 예시	<pre>[Kotlin] // 금지: prefs.edit().putString("password", password).apply() val masterKey = MasterKey.Builder(context) .setKeyScheme(MasterKey.KeyScheme.AES256_GCM).build() val prefs = EncryptedSharedPreferences.create(context, "secure_session", masterKey, EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV, EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM) // Password/PIN 은 저장하지 않고, 불가피한 단기 세션 값만 저장한다.</pre>
코드 설명	예시는 핵심 통제를 설명하기 위한 최소 구현이다. 실제 패키지·도메인·서명값은 승인된 배포 구성으로 치환하고 서버 검증과 함께 적용한다.
재점검 명령어	<pre>adb shell run-as jakhar.aseem.diva ls -la adb shell run-as jakhar.aseem.diva find . -type f adb shell find /sdcard -iname "**diva**"</pre>
기대 결과	앱 전용·외부 저장소에서 Password/PIN/카드번호/장기 Token 의 평문이 발견되지 않아야 한다. run-as 불가 시 debuggable=false 상태도 함께 기록한다.
완료 기준	앱 데이터 디렉터리·DB-Preference·외부 저장소 재점검에서 보호 대상 평문이 발견되지 않는다.
우선순위	P1

6.7 MOB-SER-009 메모리 내 중요정보 노출 방지	
진단 결과	취약
기준 위험도	5
현재 문제	민감 문자열이 immutable String 또는 불필요한 객체 복사로 메모리에 장시간 잔존할 수 있다.
보안상 영향	Root·후킹·메모리 덤프 권한을 획득한 공격자가 인증정보와 개인·금융정보를 복구할 수 있다.
개선 목표	민감 데이터의 메모리 수명과 복사를 최소화하고 키는 Android Keystore 에서 사용한다.
적용 방법	가능한 경우 CharArray/ByteArray 로 처리하고 사용 직후 덮어쓴다. 장기 Secret 을 앱 메모리에 상주시거나 로그·캐시에 복제하지 않는다.
시큐어 코딩 / 설정 예시	<pre>[Kotlin] fun authenticate(pin: CharArray) { try { val request = buildAuthRequest(pin) // 복사와 수명을 최소화 sendOverTls(request) } finally { pin.fill('\u0000') } } // Keystore 키는 export 하지 않고 Cipher 연산에 직접 사용한다.</pre>
코드 설명	예시는 핵심 통제를 설명하기 위한 최소 구현이다. 실제 패키지·도메인·서명값은 승인된 배포 구성으로 치환하고 서버 검증과 함께 적용한다.
재점검 명령어	<pre>frida-ps -U python frida-dump.py -U -n <PROCESS_NAME> rg -a "<TEST_SECRET>" .\dump</pre>
기대 결과	동일 입력 후 재수집한 dump 에서 보호 대상 데이터의 불필요한 평문 잔존이 확인되지 않아야 한다. OS·VM 특성상 완전 제거를 보장한다고 표현하지 않는다.
완료 기준	동일 테스트 데이터를 사용해 FridaDump3 로 재수집한 메모리에서 불필요한 평문 잔존이 확인되지 않는다.
우선순위	P1

6.8 MOB-SER-020 화면 내 중요정보 평문노출 방지	
진단 결과	취약
기준 위험도	5
현재 문제	카드번호·계좌번호·Password·PIN 이 화면에 전체 평문으로 표시될 수 있다.
보안상 영향	Shoulder Surfing, 화면 공유·캡처 및 악성 접근성 서비스로 중요정보가 노출될 수 있다.
개선 목표	업무에 필요한 최소 자리만 표시하고 인증 입력과 재표시를 제한한다.
적용 방법	카드·계좌번호는 앞/뒤 최소 자리만 남겨 마스킹하고 Password/PIN 입력은 보안 inputType 을 적용한다. 상세보기는 재인증과 시간 제한을 적용한다.
시큐어 코딩 / 설정 예시	<pre>[Kotlin / XML] fun maskCard(value: String): String = value.filter(Char::isDigit).let { digits -> if (digits.length < 8) "****" else "\${digits.take(4)} **** \${digits.takeLast(4)}" } <EditText android:inputType="textPassword" android:importantForAutofill="noExcludeDescendants" /></pre>
코드 설명	예시는 핵심 통제를 설명하기 위한 최소 구현이다. 실제 패키지·도메인·서명값은 승인된 배포 구성으로 치환하고 서버 검증과 함께 적용한다.
재점검 명령어	<pre>adb shell am force-stop jakhar.aseem.diva adb shell monkey -p jakhar.aseem.diva 1 adb exec-out screencap -p > screen-after.png</pre>
기대 결과	화면과 캡처 이미지에서 전체 카드번호·계좌번호·Password·PIN 이 식별되지 않아야 하며, 필요한 최소 정보만 마스킹 표시된다.
완료 기준	입력·확인·오류·복귀 화면 전체에서 원문이 불필요하게 표시되지 않고 예: 4111 **** **** 1111 형태로 표시된다.
우선순위	P1

6.9 MOB-SER-025 앱 소스코드 내 운영정보 노출 방지	
진단 결과	취약
기준 위험도	5
현재 문제	APK 는 역분석 가능하므로 하드코딩된 API Secret, Master Key, Credential 은 회수될 수 있다.
보안상 영향	운영 API 오용, 위조 요청, 계정 탈취 또는 추가 시스템 침해로 이어질 수 있다.
개선 목표	장기 Secret 과 권한 판단을 서버로 이전하고 빌드-저장소 Secret Scanning 을 적용한다.
적용 방법	APK 에는 공개되어도 되는 endpoint 와 식별자만 둔다. 사용자별 단기 토큰을 사용하고 노출 Credential 은 즉시 폐기·재발급한다.
시큐어 코딩 / 설정 예시	<pre>[Kotlin] // 금지: const val API_SECRET = "hard-coded-secret" interface ApiService { @POST("session") suspend fun createSession(@Body request: LoginRequest): SessionResponse // 서버가 단기 토큰을 발급 } // 기기 고유키가 필요하면 Android Keystore 에서 생성하고 export 하지 않는다.</pre>
코드 설명	예시는 핵심 통제를 설명하기 위한 최소 구현이다. 실제 패키지·도메인·서명값은 승인된 배포 구성으로 치환하고 서버 검증과 함께 적용한다.
재점검 명령어	<pre>jadx -d output app.apk rg -i "password secret token apikey api_key private_key" output git grep -n -i -E "password secret api[_-]?key private[_-]?key"</pre>
기대 결과	APK 와 저장소에서 실제 운영 Secret 이나 개인키가 발견되지 않아야 한다. 발견 시 삭제만 하지 말고 기존 Credential 을 폐기·재발급한다.
완료 기준	JADX·문자열 검색과 CI Secret Scan 에서 운영 Secret·개인키·Credential 이 발견되지 않는다.
우선순위	P1

6.10 MOB-SER-026 화면 강제실행에 의한 인증단계 우회 방지	
진단 결과	취약
기준 위험도	5
현재 문제	외부 노출 Activity 가 화면 도달 자체를 인증 성공으로 간주하면 ADB/Intent 로 정상 인증 흐름을 우회할 수 있다.
보안상 영향	비인가 사용자가 민감 화면·기능을 직접 실행하거나 후속 API 를 호출할 수 있다.
개선 목표	불필요한 컴포넌트 외부 노출을 차단하고 민감 화면마다 세션·권한·서버 Authorization 을 재검증한다.
적용 방법	외부 접근이 필요 없는 Activity 는 exported=false 로 설정한다. 화면 진입과 API 호출 모두 서버가 세션과 사용자 권한을 확인한다.
시큐어 코딩 / 설정 예시	<pre>[AndroidManifest.xml / Kotlin] <activity android:name=".APICredsActivity" android:exported="false" /> override fun onStart() { super.onStart() if (!session.isAuthenticated() !session.hasRole("AUTHORIZED_USER")) { finish(); return } viewModel.loadData() // 서버 API 도 Authorization 을 재검증 }</pre>
코드 설명	예시는 핵심 통제를 설명하기 위한 최소 구현이다. 실제 패키지·도메인·서명값은 승인된 배포 구성으로 치환하고 서버 검증과 함께 적용한다.
재점검 명령어	<pre>adb shell am start -n jakhar.aseem.diva/.APICredsActivity adb shell dumpsys package jakhar.aseem.diva grep -i exported</pre>
기대 결과	외부 직접 호출은 Permission Denial 또는 Activity not exported 로 실패해야 한다. 앱 내부 호출도 유효한 세션과 서버 권한이 없으면 민감정보를 반환하지 않아야 한다.
완료 기준	비로그인 상태의 직접 Activity 호출이 차단되고, 세션 만료·권한 부족 상태에서 화면과 API 모두 접근이 거부된다.
우선순위	P1

6.11 MOB-SER-042 디버그 로그 내 중요정보 노출 방지	
진단 결과	취약
기준 위험도	5
현재 문제	카드번호 등 중요정보가 Logcat 에 평문으로 기록되는 취약 코드와 동작이 확인되었다.
보안상 영향	로그 접근 권한이나 디버깅 권한을 획득한 공격자가 인증·개인정보를 수집할 수 있다.
개선 목표	민감 데이터 원문 로그를 금지하고 Release Debug/Verbose 로그를 제거한다.
적용 방법	로그에는 비민감 이벤트 ID 와 마스킹 값만 기록한다. BuildConfig.DEBUG 조건도 민감정보 기록을 허용하는 근거로 사용하지 않는다.
시큐어 코딩 / 설정 예시	<pre>[Kotlin] // 금지: Log.d("AUTH", "token=\$accessToken") if (BuildConfig.DEBUG) { Log.d("AUTH", "session request completed; requestId=\$requestId") } // 운영 오류 로그에도 카드번호·Token·Session ID 원문을 전달하지 않는다.</pre>
코드 설명	예시는 핵심 통제를 설명하기 위한 최소 구현이다. 실제 패키지·도메인·서명값은 승인된 배포 구성으로 치환하고 서버 검증과 함께 적용한다.
재점검 명령어	<pre>adb shell pidof jakhar.aseem.diva adb logcat --pid=<PID> -c adb logcat --pid=<PID> grep -E "<TEST_CARD> <TEST_TOKEN>"</pre>
기대 결과	보호 대상 테스트 값이 Logcat 에 나타나지 않아야 하며 운영상 필요한 로그는 비민감 ID 또는 마스킹 값만 포함해야 한다.
완료 기준	테스트 카드·토큰·Password 를 입력한 뒤 PID 기반 Logcat 검색에서 원문이 발견되지 않는다.
우선순위	P1

6.12 MOB-SER-043 백그라운드 화면 보호	
진단 결과	취약
기준 위험도	3
현재 문제	Recent Apps 미리보기에 교육용 중요정보가 남고 FLAG_SECURE 계열 보호 호출이 확인되지 않았다.
보안상 영향	최근 앱 스냅샷, 화면 캡처·녹화 또는 Shoulder Surfing 으로 중요정보가 노출될 수 있다.
개선 목표	민감 Activity 의 캡처·Recent Apps 노출을 제한하고 백그라운드에서 민감 View 를 비운다.
적용 방법	FLAG_SECURE 를 적용하고 onStop 에서 비민감 placeholder 로 교체한다. 복귀 시 세션·재인증 조건을 다시 확인한다.
시큐어 코딩 / 설정 예시	<pre>[Kotlin] override fun onCreate(savedInstanceState: Bundle?) { super.onCreate(savedInstanceState) window.addFlags(WindowManager.LayoutParams.FLAG_SECURE) } override fun onStop() { binding.sensitiveContainer.isVisible = false binding.privacyPlaceholder.isVisible = true super.onStop() }</pre>
코드 설명	예시는 핵심 통제를 설명하기 위한 최소 구현이다. 실제 패키지·도메인·서명값은 승인된 배포 구성으로 치환하고 서버 검증과 함께 적용한다.
재점검 명령어	<pre>adb shell am start -n jakhar.aseem.diva/<SENSITIVE_ACTIVITY> adb exec-out screencap -p > secure-screen.png adb shell input keyevent KEYCODE_APP_SWITCH</pre>
기대 결과	캡처가 차단되거나 비민감 화면만 저장되고, Recent Apps 미리보기에 카드번호·계좌·Credential 이 식별되지 않아야 한다.
완료 기준	스크린샷과 Recent Apps 미리보기에서 민감정보가 보이지 않고 복귀 후 세션 정책이 정상 동작한다.
우선순위	P1

6.13 MOB-SER-050 모바일 DeepLink 도용 제한여부

진단 결과	취약
기준 위험도	5
현재 문제	diva://app/web Custom Scheme 이 외부 호출 가능하고 URI 소유권·파라미터 검증이 제한적이다.
보안상 영향	동일 Scheme 선점, 조작 URI, 피싱 및 비의도 WebView·민감 기능 실행으로 이어질 수 있다.
개선 목표	HTTPS App Links 와 도메인 검증을 사용하고 URI 구성요소·redirect 를 Allowlist 로 제한한다.
적용 방법	autoVerify App Link 를 우선 적용하고 scheme/host/path/query 를 모두 검증한다. DeepLink 수신 후에도 세션과 서버 권한을 재확인한다.
시큐어 코딩 / 설정 예시	<pre>[AndroidManifest.xml / Kotlin] <intent-filter android:autoVerify="true"> <action android:name="android.intent.action.VIEW" /> <category android:name="android.intent.category.DEFAULT" /> <category android:name="android.intent.category.BROWSABLE" /> <data android:scheme="https" android:host="<APP_DOMAIN>" android:pathPrefix="/app" /> </intent-filter> val uri = intent.data ?: return finish() if (uri.scheme != "https" uri.host != "<APP_DOMAIN>" !uri.path.orEmpty().startsWith("/app")) finish()</pre>
코드 설명	예시는 핵심 통제를 설명하기 위한 최소 구현이다. 실제 패키지·도메인·서명값은 승인된 배포 구성으로 치환하고 서버 검증과 함께 적용한다.
재점검 명령어	<pre>adb shell am start -a android.intent.action.VIEW -d "https://<APP_DOMAIN>/app" adb shell am start -a android.intent.action.VIEW -d "https://evil.example/app" adb shell pm get-app-links jakhar.aseem.diva</pre>
기대 결과	검증된 도메인의 허용 경로만 앱으로 연결되고 잘못된 host/path/query·미인증 민감 기능 호출은 안전하게 거부되어야 한다.
완료 기준	정상 App Link 만 승인 Activity 로 연결되고 변조 host/path/query/redirect, Custom Scheme 선점 및 미인증 호출이 차단된다.
우선순위	P1

6.14 종합 개선 실행계획

우선순위	진단코드	핵심 개선통제	담당 영역	완료 기준
P1	MOB-FIN-011	단말 무결성·Root 위험평가	앱 개발 / 서버 / 보안	Nox Root 활성화·비활성 환경과 정상 단말에서 오탐·미탐을 기록하고, 서버 정책이 고위험 변조 단말의 민감 기능을 제한한다.
P1	MOB-FIN-012	악성·후킹 환경 탐지	앱 개발 / 서버 / 보안	Frida Server 실행, 후킹 패키지 설치 및 정상 환경에서 각각 재시험하여 탐지·정책·서버 이벤트 기록이 의도대로 동작한다.
P1	MOB-FIN-018	서명·APK 무결성	앱 개발 / 서버 / 보안	정상 Release APK 는 검증에 성공하고, 재서명 또는 1 byte 변조 APK 는 설치·실행 또는 서버 민감

우선순위	진단코드	핵심 개선통제	담당 영역	완료 기준
				기능 단계에서 거부된다.
P2	MOB-FIN-020	R8-리소스 축소	앱 개발 / 서버 / 보안	Release 기능 회귀 테스트를 통과하고 JADX 에서 내부 클래스-메서드 의미가 축약되며 장기 Secret 이 발견되지 않는다.
P2	MOB-FIN-021	Release Anti-Debug	앱 개발 / 서버 / 보안	Release APK 가 debuggable=false 이고 adb jdwp/JDB 연결이 불가능하며 탐지 이벤트가 서버 정책에 반영된다.
P1	MOB-SER-008	로컬 중요정보 최소화	앱 개발 / 서버 / 보안	앱 데이터 디렉터리-DB-Preference-외 부 저장소 재점검에서 보호 대상 평문이 발견되지 않는다.
P1	MOB-SER-009	메모리 평문 수명 최소화	앱 개발 / 서버 / 보안	동일 테스트 데이터를 사용해 FridaDump3 로 재수집한 메모리에서 불필요한 평문 잔존이 확인되지 않는다.
P1	MOB-SER-020	화면 마스킹	앱 개발 / 서버 / 보안	입력-확인-오류-복귀 화면 전체에서 원문이 불필요하게 표시되지 않고 예: 4111 **** * 1111 형태로 표시된다.
P1	MOB-SER-025	Secret 서버 이전	앱 개발 / 서버 / 보안	JADX-문자열 검색과 CI Secret Scan 에서 운영 Secret-개입키-Credential 이 발견되지 않는다.
P1	MOB-SER-026	컴포넌트-서버 권한 검증	앱 개발 / 서버 / 보안	비로그인 상태의 직접 Activity 호출이 차단되고, 세션 만료-권한 부족 상태에서 화면과 API 모두 접근이 거부된다.
P1	MOB-SER-042	민감 로그 제거	앱 개발 / 서버 / 보안	테스트 카드-토큰-Password 를 입력한 뒤 PID 기반 Logcat 검색에서 원문이 발견되지 않는다.
P1	MOB-SER-043	FLAG_SECURE-Snapshot 보호	앱 개발 / 서버 / 보안	스크린샷과 Recent Apps 미리보기에서 민감정보가 보이지 않고 복귀 후 세션 정책이 정상 동작한다.
P1	MOB-SER-050	검증된 App Links-URI Allowlist	앱 개발 / 서버 / 보안	정상 App Link 만 승인 Activity 로 연결되고 변조 host/path/query/redirect,

우선순위	진단코드	핵심 개선통제	담당 영역	완료 기준
				Custom Scheme 선점 및 미인증 호출이 차단된다.

7. 최종 결론

DIVA DeepLink 를 대상으로 MOBILE 13 개 점검항목을 진단한 결과, 전체 항목에서 보안통제 미흡 또는 취약 동작이 확인되었다.

특히 루팅·동적 분석 환경에 대한 방어, 앱 무결성 검증, 중요정보 보호, 인증 및 딥링크 검증 기능을 우선 개선할 필요가 있다.

최종 판정: 취약 — 개선조치 후 재점검 필요. 관련 보호통제를 적용한 후 본 보고서와 동일한 절차로 취약점이 해소되었는지 재점검할 것을 권고한다. DIVA DeepLink 는 교육용 애플리케이션이므로 본 결과를 실제 전자금융 서비스의 운영 적합성으로 확대 해석해서는 안 된다.